

Certified lower bounds for correlation clustering on large sparse graphs

Quang-Vinh Dang^{1*}

^{1*}British University Vietnam, Hung Yen, Vietnam.

Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Abstract

Correlation clustering (equivalently, cluster editing) has strong approximation algorithms and strong heuristics, but on large graphs nobody can say how far a given clustering is from optimal: exact solvers stop at a few hundred vertices, and the lower bounds used at scale so far, packings of bad triangles, certify only factors of 1.17 to 1.90 on our graphs. We compute lower bounds on sparse unweighted instances with up to 10^6 vertices and certify every one of them. Optimal clusters have diameter at most two, so the metric LP, its strengthening by star and local subgraph inequalities, and every packing of induced stars can be restricted to the $O(\sum_v \deg(v)^2)$ pairs at distance at most two. On these pairs we reimplement the star-packing local search of the KaPoCE branch-and-bound with memory linear in the graph and the packing, and add a dual block-coordinate ascent that turns the relaxation into an anytime bound. Certificates are verified against the raw instance file, in exact arithmetic, by a short checker that shares no code with the solver; the inequality it evaluates is proved in Lean 4. On the PACE 2021 exact track, under the stopping rule of the branch-and-bound, the reimplement proves optimality on 83 of the 173 solved instances, against 79 for the published root bound; run for 60s, the same heuristic proves it on 122, and the block LPs on 7 more that longer packing does not close. The checked bounds improve the published root bounds of 22 of the 27 open instances. On all 27 SNAP graphs of our study, with up to 1,134,890 vertices, we obtain checked bounds; on the 23 with an archived clustering they certify it within 0.22% to 20.9% (median 3.1%), where triangle packings leave 17% to 90%. The dense implementation of the branch-and-bound runs on seven of these graphs, exceeds our best bound on 2 of them by at most 0.4%, and its exported packings pass the same checker.

Keywords: correlation clustering, cluster editing, lower bounds, linear programming, dual ascent, certificates, verified computation

1 Introduction

In *correlation clustering* [1] the input is a complete graph whose pairs are labelled *similar* (+) or *dissimilar* (−), and the task is to partition the vertices so that the number of disagreements, + pairs between clusters plus − pairs inside clusters, is minimum. The number of clusters is free. Equivalently, given the positive graph $G^+ = (V, E^+)$, one adds or deletes the fewest edges that turn G^+ into a disjoint union of cliques (*cluster editing*). The problem is APX-hard [2]. It arises in entity resolution, community detection and the consolidation of inconsistent pairwise information.

Two lines of work have progressed separately. Approximation ratios for complete graphs have fallen from 3 for Pivot [3] through 2.06 by rounding the metric LP [4] to 1.485 with the cluster LP [5] and below [6]. In practice, engineered heuristics such as KaPoCE [7], the winner of the PACE 2021 challenge [8], and multilevel memetic solvers [9] find very good solutions on large graphs (for the latter, on sparse signed graphs with millions of vertices, whose objective differs from ours; Section 2). What is missing on such graphs is a *certificate*: a lower bound on the optimum, checkable independently of the solver, that tells how far a given clustering can be from optimal. Exact branch-and-bound solves instances with a few hundred vertices [10], and its strong packing bounds are implemented on a dense $n \times n$ matrix. Solvers for the metric LP of correlation clustering on complete graphs reach about 10^4 vertices [11, 12]. The combinatorial bounds that do scale, packings of bad triangles, certify solutions only within factors of 1.17 to 1.90 on the graphs of our study; the LP bounds derived from strong triadic closure [13] are supported on the same wedges.

This paper is about the lower-bound side. We compute lower bounds for correlation clustering on sparse graphs with up to 1,134,890 vertices and 2,987,624 edges, export them as certificates that an independent program checks in exact arithmetic against the raw instance file, and use them to certify the solutions of our own and of other solvers. The primal methods are secondary; they are included because a bound is only as useful as the solutions it is compared with, and because the certificate yields a decomposition of the optimality gap that guides a local search.

Contributions.

The paper contributes an implementation that makes the strongest combinatorial lower bound known for cluster editing, the star packing of the KaPoCE branch-and-bound, run on graphs with 10^6 vertices; a certificate format with an independent exact checker; and a computational study of how strong each bound is and where its strength comes from.

1. **Distance-two support** (Section 3.1). Optimal clusters have diameter at most two in G^+ , so every pair at distance at least three is separated in every optimal clustering; the KaPoCE branch-and-bound uses this to forbid such pairs [10]. We state the consequence for relaxations: the metric LP restricted to the $O(\sum_v \deg(v)^2)$

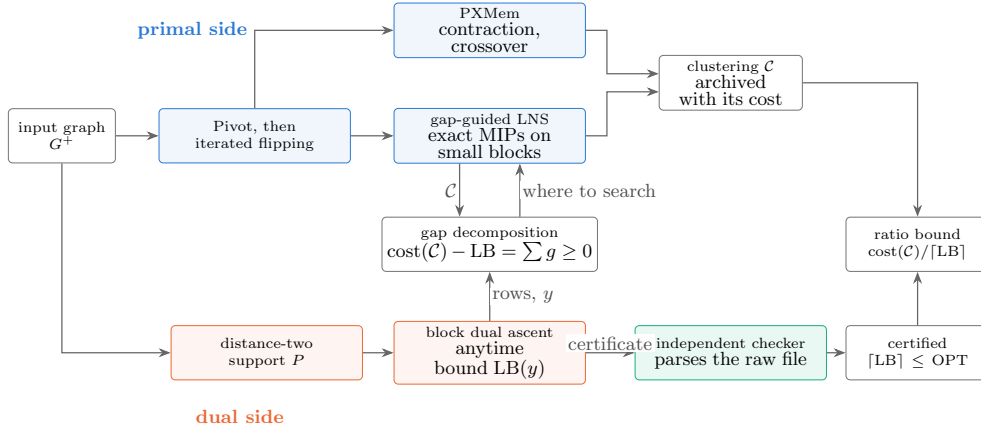


Fig. 1 Overview. The dual side (orange) builds the distance-two support and a lower bound from a star packing on it, found by local search and improved by block dual ascent; the certificate is rechecked by an independent program in exact arithmetic against the raw instance. The primal side (blue) computes clusterings: Pivot and iterated flipping give a start, and either the gap-guided LNS of CertiFlip or the memetic search PXMem improves it. The gap decomposition splits $cost(C) - LB$ into non-negative local terms and tells the LNS where to search.

pairs at distance at most two, with the triangle inequalities through distance-three pairs kept as covering rows, is a relaxation whose feasible points extend to the full metric LP at the same cost (Theorem 3), and star and local subgraph inequalities remain valid on it. The observation is simple and its value is enabling: the bounds, their dual solutions and their certificates are stored on these pairs. Every packing of induced stars lives there as well.

2. **Star local search on the support** (Section 3.2). The root bound of the KaPoCE branch-and-bound [10] is a star packing found by a local search that extends the P_3 -packing search of Gottesbüren et al. [14]; it is implemented on a dense $n \times n$ matrix. We reimplement the same heuristic on the sparse graph, with a hash map over the used pairs in place of the matrix, so that its memory is linear in the graph plus the packing, and write its stars directly as certificate rows. What is new is the scale: it runs on graphs with 10^6 vertices, where the dense implementation does not fit into memory. It is not a better heuristic. With the stopping rule of the branch-and-bound, it proves optimality on 83 of the 173 solved PACE exact instances in a median of 0.5 s, against 79 for the published root bound; run for 60 s, it proves optimality on 122. The published root bounds are thus far from what the same heuristic reaches with a few seconds more.
3. **Anytime block-dual bound** (Section 3.2). We compute a lower bound by dual block-coordinate ascent, a relax-and-cut scheme: each step solves the LP of one vertex block exactly by cutting planes, with costs equal to the current reduced costs. The bound is valid after every step and never decreases, and its state is one reduced cost per pair plus the rows with positive multipliers (Theorem 5). Any star or triangle packing is a feasible start. After a strong star packing, and at an equal budget, it pays on the SNAP graphs only where short cycles rather than stars carry

the bound (`ca-GrQc`, `roadNet-PA`; Section 6.3); on the PACE exact track it closes 7 instances that the star local search does not close with five times the budget or with five seeds.

4. **Exactly checked certificates** (Section 4). A certificate lists the rows with positive multipliers. A checker of about 633 lines that shares no code with the solver parses the raw instance file, verifies that it is the instance the certificate was computed for (SHA-256 of the file and of the canonical edge list), checks the validity of every row by enumeration or by a closed form, and evaluates the bound in integer arithmetic. The inequality it evaluates, given valid rows, is proved in Lean 4; the validity procedures are tested, not verified. The format is specified independently of the code, and the checker also accepts the packings of the KaPoCE branch-and-bound, exported from its code. We archive every certificate behind every reported bound.
5. **Computational study** (Section 6). On the PACE 2021 exact track, the checked bounds improve the published root bounds of 22 of the 27 open instances. On all 27 SNAP graphs we obtain checked bounds; on the 23 with an archived clustering, the largest checked bound certifies it within 0.22% to 20.9% (median 3.1%), where triangle packings leave 17% to 90%. We report the spread of the star local search over five seeds and its progress over time, compare it at equal budget with the block LPs, and compare with the triangle packings, the metric LP on the support with triangle or star rows, a multicut dual solver, and the root bound of the branch-and-bound where it runs.

The certificate also yields a decomposition of the optimality gap into non-negative local terms (Section 5), which the primal method CertiFlip uses to choose the neighbourhoods of a large-neighbourhood search; this choice is not evaluated separately, and we do not count it as a contribution. A second primal solver, PXMem (Appendix C), recombines two clusterings exactly block by block of their overlap; its comparison with KaPoCE under a protocol fixed in advance, which finds no significant difference at 600s, is in Appendix C.5. The upper bounds against which the certificates are measured are the best clusterings of all solvers, KaPoCE included.

Relation to certified computation.

Certifying algorithms [15] return a witness with every output. Safe bounds from floating-point LP solutions [16, 17], checkable certificates for mixed-integer programming [18] and proof logging in SAT and pseudo-Boolean solving [19, 20] pursue the same goal. Our certificates are simpler than general MIP certificates because every row is local: its validity is decided by enumerating the partitions of at most nine vertices or by a closed form (Section 4 compares them with VIPR and pseudo-Boolean proof logging).

Development and evaluation data.

The methods were developed on fifteen SNAP graphs and the PACE 2021 exact track. Before the final primal experiments we fixed a held-out set of twelve further SNAP graphs and an analysis plan for the comparison with KaPoCE (Appendix C.5). The bound procedures were not evaluated out of sample: the star local search was written after results on all graphs were known and was tried on seven held-out graphs during

its development, so we report the SNAP bounds as one benchmark and keep the distinction between development and held-out graphs only as a description. Appendix D records which data informed which design decision.

2 Preliminaries and related work

Let $G^+ = (V, E^+)$ with $n = |V|$ and $m = |E^+|$; every pair not in E^+ is negative. For a clustering $\mathcal{C}$ let $x_{uv}^{\mathcal{C}} = 1$ if u, v are in different clusters and 0 otherwise. Then

$$\text{cost}(\mathcal{C}) = \sum_{uv \in E^+} x_{uv}^{\mathcal{C}} + \sum_{uv \notin E^+} (1 - x_{uv}^{\mathcal{C}}).$$

$N(v)$ denotes the positive neighbourhood of v and $d(u, v)$ the distance in G^+ . The *metric LP* [2, 3] relaxes x to $[0, 1]^{\binom{V}{2}}$ subject to the triangle inequalities $x_{uv} \leq x_{uw} + x_{wv}$ for all triples. A *bad triangle* is a triple u, w, v with $uw, wv \in E^+$ and $uv \notin E^+$, that is, an induced path on three vertices (P_3), also called a conflict triple; every clustering makes at least one mistake on it. Packings of bad triangles and P_3 packings are therefore the same bound and differ only in the heuristic that finds them. An *induced star* with centre v and $k \geq 2$ leaves is a set of k pairwise non-adjacent neighbours of v ; every clustering makes at least $k - 1$ mistakes on its $k + \binom{k}{2}$ pairs.

Approximation.

Pivot repeatedly picks a uniformly random unclustered vertex and clusters it with its unclustered positive neighbours; its expected cost is at most three times the fractional packing LP of bad triangles [3]. LP-based pivoting joins v to the pivot u with probability $1 - f(x_{uv})$; $f(x) = x$ gives 2.5 [3], and the rounding functions of Chawla et al. [4] give 2.06. Both guarantees hold against the value of *any* feasible metric x , a fact we use below. Further improvements use Sherali–Adams hierarchies [21], preclustering [22] and the cluster LP [5, 6, 23]; the best combinatorial ratio, $2 - 2/13$, comes from a local search with flips [24]. For parallel algorithms see [25, 26], for streaming and dynamic ones [27–29], and for weighted general graphs [30]; Böcker and Baumbach [31] survey cluster editing.

Lower bounds.

The classical scalable bounds are packings of bad triangles, greedy or fractional; the latter approximates the covering LP over bad triangles and can be computed by the multiplicative-weights scheme of Garg and Könemann [32, 33]. Veldt [13] derives LP relaxations from strong triadic closure that are supported on the open wedges of G^+ , like our relaxation, and yield fast combinatorial approximation algorithms, among them MatchFlipPivot, together with lower bounds; our relaxation keeps the full metric LP on these pairs and adds stronger rows. The metric LP of correlation clustering on complete graphs has been solved on graphs with about 10^4 vertices by projection methods [11, 12]. The Project and Forget scheme [34], which also keeps only active triangle rows, reaches larger sparse instances of correlation clustering on general graphs, where pairs without an edge carry no cost; that objective differs from ours

in the same way as the multicut objective below. Exact branch-and-bound for cluster editing [10, 35, 36] uses packings of stars and of induced paths on three vertices as lower bounds. Bläsius et al. [10, Theorem 1] show that for any family of pairwise non-conflicting integer cost functions whose absolute values sum to at most the cost of every pair, the sum of their optima is a lower bound, and evaluate P_3 , star and $K_{a,b}$ packings. Their packing local search extends the P_3 -packing search of Gottesbüren et al. [14] by leaf and merge moves and a relaxed stopping rule, and gives a star an integer multiplicity on weighted instances. They also mark the pairs at distance at least three as forbidden (the “distance-three rule”), while Bastos et al. [37] observe that these pairs are separated in every optimal solution. Packings of P_3 s are also the lower bound above which parameterised algorithms for cluster editing are analysed [38, 39]. For the clique partitioning problem, the weighted maximisation form, facets and cutting planes go back to Grötschel and Wakabayashi [40, 41]; later work removes redundant triangle rows [42–44], adds chorded-cycle facets [45], uses Lagrangian relaxation with pegging tests [46], and tightens bounds with constraints on subnetworks [47]. Exact approaches via weighted MaxSAT [48] and kernelisation [49, 50] solve smaller instances to optimality.

For the multicut problem on sparse graphs, where missing pairs carry no cost, dual decomposition and message passing give fast lower bounds [51–53], and persistency criteria fix variables that take the same value in every optimal solution [54, 55]. Our block-dual ascent is closest in spirit to block-coordinate ascent for dense graphical models [56]: each block is solved to optimality with the other multipliers fixed. In operations-research terms it is a relax-and-cut scheme [57], in which rows are separated dynamically and dualised, with the safe bound of Neumaier and Shcherbina [16] for arbitrary multipliers. Like every block-coordinate method it can stop at a point that is not optimal for the whole LP when the blocks do not cover the graph. First-order LP methods such as PDLP [58] solve large LPs globally, and their dual iterates could be certified in the same way; we did not try them. These multicut methods are not designed for correlation clustering on complete graphs, where the $\binom{n}{2} - m$ negative pairs all carry cost, but the distance-two support reduces the complete instance to a sparse one. For every clustering $\mathcal{C}$, the multicut objective on the graph (V, P) with weight $+1$ on E^+ and -1 on N_2 counts the positive pairs between clusters and subtracts the negative pairs of P between clusters, so it differs from $\text{cost}(\mathcal{C})$ by a constant $(|N_2|)$ plus the number of negative pairs outside P inside clusters, which is non-negative. Hence any multicut lower bound on (V, P) , plus $|N_2|$, is a valid lower bound for correlation clustering, and dual solvers such as RAMA [52] can be run on the support. We ran RAMA (Table 7); the following argument shows why such bounds cannot exceed a weak relaxation. Dual methods based on cycle inequalities give at most the value of the multicut LP on (V, P) with all cycle inequalities. Every feasible point of LP_P extends to a metric on V (Theorem 3(b)) and therefore satisfies all cycle inequalities of (V, P) , and the two objectives agree up to the constant $|N_2|$. This also holds for solvers that first add edges of weight zero, as RAMA does, since a metric on V satisfies the cycle inequalities of every graph on V ; dual methods with further inequalities, such as odd wheels, are not covered by the argument. So these bounds are at most the value of LP_P with triangle rows, which on the SNAP graphs is far below our checked bounds

wherever it converged (Table 7); the RAMA bounds are below it on every graph where it converged.

Heuristics.

Local search with vertex moves [59], Louvain and Leiden with the constant Potts model at resolution $1/2$, which optimise the same objective [60–62], heuristics for clique partitioning [63–65], local search for multicut [66] and fusion moves [67] are the common primal methods. KaPoCE combines reductions, local search and an evolutionary phase [7]. SCC [9] is a multilevel memetic solver for correlation clustering on sparse signed graphs, in which pairs without an edge carry no cost; its objective therefore differs from the complete-graph objective studied here (Section 6.1). Twin (critical-clique) reductions and their weighted variants are standard in parameterised algorithms and practical tools [49, 68–73].

3 Lower bounds on the distance-two support

3.1 Distance-two relaxations

Lemma 1 *Let $\mathcal{C}$ be an optimal clustering, $S \in \mathcal{C}$ and $v \in S$. Then $|N(v) \cap S| \geq (|S| - 1)/2$. Consequently any two vertices of S are adjacent or have a common neighbour in S . In particular every pair at distance at least three is separated.*

Proof Moving v to a singleton changes the cost by $|N(v) \cap S| - (|S| - 1 - |N(v) \cap S|)$, which must be non-negative. For non-adjacent $u, v \in S$ the sets $N(u) \cap S$ and $N(v) \cap S$ lie in $S \setminus \{u, v\}$, which has $|S| - 2$ elements, and have total size at least $|S| - 1$, so they intersect. $\square$

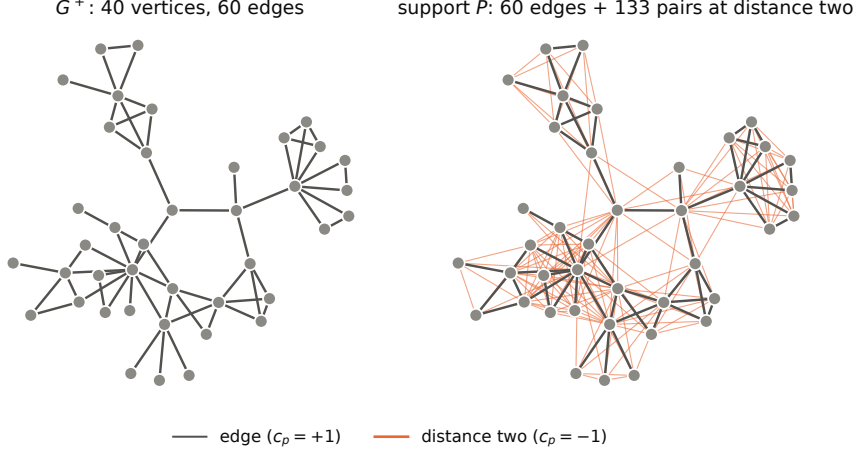
Lemma 1 is folklore; its consequence for distance-three pairs is noted by Bastos et al. [37] and used to forbid pairs in the branch-and-bound of Bläsius et al. [10] (their distance-three rule), so that every bound of that solver is implicitly supported on the pairs at distance at most two. The same pairs carry the strong-triadic-closure LPs of Veldt [13], and the preclustering of Cohen-Addad et al. [22], used by Cao et al. [5], restricts the LP variables to a sparse set of admissible pairs at a loss of a factor $1 + \varepsilon$. The restriction below is the exact, loss-free counterpart for the metric LP and its strengthenings; Theorem 3 states it formally, and its value is enabling rather than conceptual: bounds, dual solutions and certificates can be stored on these pairs.

Let $N_2 = \{uv \notin E^+ : N(u) \cap N(v) \neq \emptyset\}$, $P = E^+ \cup N_2$ and $F = \binom{V}{2} \setminus P$ (the *far* pairs). Note $|P| \leq m + \sum_w \binom{\deg(w)}{2}$. Figure 2 shows the support on a small neighbourhood of a collaboration network: already there, 587 of the 780 pairs (75%) are far and disappear from the relaxation.

Definition 2 (distance-two relaxation) LP_P has a variable $x_e \in [0, 1]$ for every $e \in P$ and minimises $\sum_{e \in E^+} x_e + \sum_{e \in N_2} (1 - x_e)$ subject to

$$x_{uv} \leq x_{uw} + x_{wv} \quad \text{for all } u, v, w \text{ with } uv, uw, wv \in P, \quad (1)$$

$$x_{uv} + x_{wv} \geq 1 \quad \text{for all } u, v, w \text{ with } uw, wv \in P, uv \in F. \quad (2)$$



The other 587 of the 780 pairs are far and are fixed to separated.

Fig. 2 The distance-two support on a 40-vertex neighbourhood of **ca-GrQc**. Left: the positive graph. Right: the support P adds the non-adjacent pairs with a common neighbour (orange); every other pair is far, separated in every optimal clustering (Lemma 1), and carries no variable.

Theorem 3 Let LP_{std} denote the value of the metric LP.

- (a) $LP_{\text{std}} \leq LP_P \leq \text{OPT}$.
- (b) If x is feasible for LP_P , then $\bar{x}$, defined as x on P and 1 on F , is feasible for the metric LP and has the same cost.
- (c) Let $\mathcal{A}$ be a randomised rounding algorithm with $\mathbb{E}[\text{cost}(\mathcal{A}(\bar{x}))] \leq \alpha \text{cost}(\bar{x})$ for every feasible metric $\bar{x}$. Applied to an optimal solution of LP_P it is an α -approximation. This holds in particular for $\alpha = 2.5$ (ACN) and $\alpha = 2.06$ (CMSY). With pivot-based rounding, only pairs in P can be joined, so the rounding takes $O(n + |P|)$ time.

Proof (b) Consider a triple u, v, w and the number of far pairs among uv, uw, vw . With none, all three triangle inequalities are rows (1). With exactly one, say uv , the inequality $\bar{x}_{uv} = 1 \leq x_{uw} + x_{wv}$ is row (2). The other two inequalities, such as $x_{uw} \leq 1 + x_{wv}$, hold because $x \leq 1$. With two or three far pairs, each inequality has a right-hand side of at least 1. Far pairs are negative and $\bar{x} = 1$ on them, so they contribute 0 to the cost. (a) The first inequality follows from (b). For the second, by Lemma 1 an optimal clustering separates all far pairs. Its cut vector restricted to P therefore satisfies (1) and (2), because it is a metric with $x = 1$ on F , and it has cost OPT . (c) follows from (a) and (b): the expected cost is at most $\alpha \text{cost}(\bar{x}) = \alpha LP_P \leq \alpha \text{OPT}$. Pivot-based rounding joins v to the pivot u with probability $1 - f(x_{uv})$, and $f(1) = 1$ for both rounding functions. $\square$

The rows (1)–(2) are generated lazily. A violated row needs $x_{uw} + x_{wv} < 1$, so separation around a vertex w only has to look at pairs of “close” neighbours in P , which keeps it fast in practice.

Stronger inequalities.

The integrality gap of the metric LP is at least 2 (stars attain it asymptotically) and at most 2.06 [4]. We add two families that are valid for every clustering separating all far pairs, and hence for every optimal clustering.

Proposition 4 *Let $\mathcal{C}$ be a clustering in which no far pair shares a cluster.*

- (i) *(star inequalities; the case $|S| = 1$ of the 2-partition inequalities of Grötschel and Wakabayashi [40, 41], with the far pairs dropped from the second sum, which makes them stronger and valid only for far-separating clusterings) For every v and $T \subseteq V \setminus \{v\}$, with $y = 1 - x^{\mathcal{C}}$,*

$$\sum_{t \in T} y_{vt} - \sum_{\{t, t'\} \subseteq T, tt' \in P} y_{tt'} \leq 1.$$

- (ii) *(local subgraph inequalities) For every $H \subseteq V$,*

$$\sum_{e \in E^+(H)} x_e^{\mathcal{C}} + \sum_{e \in N_2(H)} (1 - x_e^{\mathcal{C}}) \geq \text{OPT}(H),$$

where $\text{OPT}(H)$ is the optimum of the instance induced by H and $E^+(H)$, $N_2(H)$ are the pairs of E^+ and N_2 inside H . More strongly, the right-hand side may be the minimum over the clusterings of H that separate the far pairs inside H , which can exceed $\text{OPT}(H)$; the checker accepts every row with this stronger right-hand side (Section 4). These are the subnetwork constraints of Belyi et al. [47] in the minimisation form. With multipliers $y = 1$ on pair-disjoint rows, or with integer multipliers whose coefficients sum to at most 1 in absolute value on every pair, the bound (3) below is the packing bound of Bläsius et al. [10, Theorem 1]; the LP form adds fractional multipliers and the combination with the metric rows.

Proof (i) If the cluster of v contains k vertices of T , the first sum is k and the second is at least $\binom{k}{2}$. Pairs inside T that are not in P are far, so they are separated and contribute 0 to the second sum. Finally $k - \binom{k}{2} \leq 1$. (ii) Restricted to H , $\mathcal{C}$ is a clustering of H . Its mistakes inside H number at least $\text{OPT}(H)$. The far pairs inside H are separated and cost nothing, so all its mistakes are counted on the left-hand side. $\square$

The local subgraph inequalities are local cuts in the sense of Applegate et al. [74]: valid inequalities obtained by solving a small subproblem exactly. Bad triangles ($\text{OPT} = 1$) and induced stars with k independent leaves ($\text{OPT} = k - 1$) are local subgraph inequalities. So are chordless cycles C_k with odd k , for which $\text{OPT}(C_k) = \lceil k/2 \rceil$ while $x = 1/2$ on the cycle edges gives $k/2$; rounding the total bound up recovers one such half unit, the row recovers one for every cycle. We separate stars greedily around each vertex. For (ii) we grow H ($|H| \leq 8$) by breadth-first search along fractional pairs and compute $\text{OPT}(H)$ by enumerating the at most 4140 set partitions of H .

3.2 Anytime dual bounds

Write the relaxation as $\min\{K + c^\top x : Ax \geq b, 0 \leq x \leq 1\}$ over $x \in \mathbb{R}^P$, with $c_e = 1$ on E^+ , $c_e = -1$ on N_2 and $K = |N_2|$. The rows of (A, b) are any valid inequalities from Section 3.1.

Theorem 5 *For every $y \geq 0$ let $r = c - A^\top y$ and*

$$\text{LB}(y) = K + b^\top y + \sum_{e \in P} \min\{0, r_e\}. \quad (3)$$

- (a) $\text{LB}(y) \leq \text{OPT}$.
- (b) Let $B \subseteq V$, let R_B be a set of rows supported on pairs with both endpoints in B , and let $\bar{r}$ be the reduced costs after removing the contribution of the current multipliers on R_B . Replace these multipliers by an optimal dual solution of $\min\{\bar{r}^\top x_B : A_B x_B \geq b_B, 0 \leq x_B \leq 1\}$. This does not decrease LB .
- (c) If $B = V$ and R_B consists of all rows of LP_P together with all rows that carry a positive multiplier, one block step gives $\text{LB} \geq \text{LP}_P$, with equality if R_B has no further rows.

Proof (a) Let $\mathcal{C}$ be optimal. By Lemma 1 it separates far pairs, so $\text{OPT} = K + c^\top x^{\mathcal{C}}$ with $x^{\mathcal{C}} \in \{0, 1\}^P$ satisfying $Ax^{\mathcal{C}} \geq b$. Then $\text{OPT} = K + r^\top x^{\mathcal{C}} + y^\top Ax^{\mathcal{C}} \geq K + \sum_e \min\{0, r_e\} + y^\top b$. (b) By LP duality the block LP equals $\max_{y_B \geq 0} b_B^\top y_B + \sum_{e \in P_B} \min\{0, (\bar{r} - A_B^\top y_B)_e\}$. The terms of (3) outside the block do not change, and the old multipliers are feasible for this maximisation. (c) With $B = V$ and every row with a positive multiplier in R_B , removing their contribution gives $\bar{r} = c$, so the block LP is LP_P with the rows of R_B added, whose value is at least LP_P and equal to it if no row is added; by LP duality with the box constraints, LB equals this value. If a row with a positive multiplier were left outside R_B , $\bar{r} \neq c$ and the bound could be below LP_P . $\square$

Part (b) needs an optimal block dual. When a block LP stops at its time limit, its duals are only feasible; the bound computed with them is still valid by part (a), but it may be smaller than before. The implementation therefore evaluates the new bound and keeps a block step only if the bound does not decrease, so the bound is monotone in every case.

Warm start by star packings.

Every pair-disjoint family of induced stars (a centre v with $k \geq 2$ pairwise non-adjacent positive neighbours) is a feasible dual solution: put $y = 1$ on the corresponding local subgraph inequalities of Proposition 4(ii), which in x -form read $\sum_t x_{vt} - \sum_{tt'} x_{tt'} \geq (k-1) - \binom{k}{2}$. Then $\text{LB}(y)$ equals the packing value $\sum (k-1)$. Indeed, a centre pair has $c_e = 1$ and coefficient 1, and a leaf pair has $c_e = -1$ and coefficient -1 , so every packed pair has reduced cost 0. Unpacked pairs keep $r_e = c_e$. Hence $\text{LB} = K + \sum_S ((k_S - 1) - \binom{k_S}{2}) - |N_2 \setminus \text{packed}|$, and $K - |N_2 \setminus \text{packed}| = \sum_S \binom{k_S}{2}$. Fractional packings of bad triangles, as computed by the Garg–Könemann scheme [32, 33], are installed in the same way on the rows $x_{uw} + x_{wv} - x_{uv} \geq 0$. Packings are the bounds used inside the KaPoCE branch-and-bound algorithm [10]. Its implementation stores

the instance as a dense matrix, so it applies only to graphs with at most about $2.5 \cdot 10^4$ vertices on our machines. Every pair of an induced star is an edge or a pair of leaves with a common neighbour, the centre, so every star packing lives on the support P , and a packing heuristic needs no dense matrix. We use two.

Star local search. This is our implementation, on the sparse graph, of the star-packing heuristic of the KaPoCE branch-and-bound [10], which extends the P_3 -packing search of Gottesbüren et al. [14]; it is a separate implementation that shares no code with KaPoCE. The moves and the stopping rule below are those of Bläsius et al. [10]; we add a minimum running time, the sparse representation and the direct export of the stars as certificate rows, and make no claim that the search itself is better. Pairs are represented by their endpoints, and a hash map from the pairs used by the packing to their star replaces the matrix, so the memory is $O(n + m)$ plus the number of used pairs. The start processes the vertices by decreasing degree; for a vertex v it colours the conflict graph on the neighbours t with a free pair vt (two neighbours conflict if they are adjacent or their pair is used) greedily in reverse degeneracy order, and every colour class of at least two neighbours becomes a star centred at v . The local search visits the stars in random order and tries three moves. A *merge* joins two stars with the same centre whose leaves are pairwise non-adjacent through free pairs (+1). An *exchange* removes one leaf ℓ from a star, or the whole star if it has two leaves, which frees the pairs of ℓ ; it lists the candidates that use a freed pair, namely new stars with two leaves and extensions of existing stars by one leaf, and applies two candidates through different freed pairs if they are compatible (+1). Otherwise a *plateau move* applies one candidate, with probability 0.8 one whose pairs lie in the fewest induced paths on three vertices and otherwise a random one, which keeps the value. Rounds are repeated until they are more than five times as many as the rounds that improved the packing, but at least for a given time, and at most until the time limit. The stars are written out directly as certificate rows, so neither the search nor the certificate needs the support. A star with k leaves occupies $k + \binom{k}{2}$ pairs; on graphs with very large degrees we therefore cap the number of leaves per star, which loses at most one unit of bound per split star.

Ruin and recreate. Our first packing, which CertiFlip uses as its warm start, builds a greedy packing on the sparse support, processing centres by decreasing degree, and improves it by ruin and recreate. (On dense instances large stars waste pairs, since a star with k leaves uses $k + \binom{k}{2}$ pairs for a value of $k - 1$. There we also try a packing that first packs bad triangles maximally and then extends stars on free pairs, as well as the fractional triangle packing, and install the best of the candidates.) Each step picks a random vertex, removes all stars centred in a random connected region of about 12 vertices around it, and repacks the freed pairs greedily in a random order. A step is kept iff the value does not decrease.

Either packing is installed as the initial multipliers. Subsequent block steps release the packed rows that lie inside a block and re-optimize them fractionally together with newly separated rows. By Theorem 5(b), the final bound is never below the packing value.

The bound (3) needs only the vector r and the scalar $b^\top y$. We store the rows with positive multipliers so that a later block containing them can re-optimize them. Blocks

of the same partition are independent. In a *sweep* we compute a partition into parts of about 2500 vertices with METIS [75] on a randomly relabelled copy of G^+ , so that successive sweeps cut different pairs. Each part is solved with the dual simplex method of HiGHS [76], with lazily separated triangle, star and local subgraph rows. After the sweeps stall we switch to *gap-guided blocks*, grown around the vertices with the largest share of the gap of Section 5. Because (3) is valid for any $y \geq 0$, the bound remains correct even when a block LP hits its time limit. We only clamp the dual values to be non-negative, and reject a block step that would lower the bound.

Floating point.

The block LPs are solved in double precision, so the multipliers y are inexact and the rows they carry are generated by floating-point separation. Neither affects validity: (3) holds for every $y \geq 0$ and every set of valid rows. The value the solver reports is nevertheless not the value we publish. Section 4 recomputes it from the exported rows and multipliers in exact arithmetic, after checking every row, and all bounds in Section 6 are these recomputed values.

4 Certificates and their independent check

Every lower bound reported in Section 6 is the output of a separate checker, not of the solver. This section specifies what a certificate contains, what the checker verifies, and what has to be trusted.

Contents.

A certificate is a compressed NumPy archive with

- the rows with positive multiplier, in compressed sparse row form: for row i the pairs $p = \{u, v\}$ of its support, integer coefficients a_{ip} and an integer right-hand side b_i , in the x -form $a_i^\top x \geq b_i$;
- the multipliers $y_i \geq 0$ as double-precision numbers;
- the identity of the instance: the name and relative path of the raw file as distributed by PACE or SNAP, its SHA-256, its format, the number of vertices n and edges m , and the SHA-256 of the canonical edge list, that is, of n followed by the lexicographically sorted pairs (u, v) with $u < v$ as 64-bit integers.

The vertex numbering is fixed by the instance definition: a PACE file numbers the vertices $1, \dots, n$; in a SNAP edge list the vertices are the identifiers that occur in a kept edge, numbered in increasing order, a signed list keeps only its positive rows, and self-loops and duplicate edges are dropped. The clustering whose cost is reported is archived next to the certificate, one label per vertex in the same numbering.

The check.

The checker (`experiments/check_certificate.py`, 633 lines of Python with Numba kernels) imports nothing from the solver package. Given the raw file and the certificate it

1. parses the raw file with its own reader and rejects the certificate unless it records n , m , the file hash and the edge-list hash and all four agree with the file. When it checks a directory, the instance file named in a certificate must be listed with the same hash in the committed manifest of instance files, and the parser settings of that file (format and sign column) are taken from a committed instance table, never from the certificate, so a certificate can neither choose the instance it is checked against nor how that instance is read;
2. checks that every pair of every row is in P , i.e. is an edge or has a common neighbour, by intersecting sorted adjacency lists;
3. checks that no row contains a pair twice and that the absolute values of the coefficients of every row sum to less than 2^{61} , so that every partial sum below is exact in 64-bit integers;
4. checks that every row is valid for every clustering that separates the far pairs. A row on at most nine vertices is checked by enumerating all partitions of its vertex set (at most 21,147, as restricted growth strings) that separate its far pairs and taking the minimum of the integer left-hand side. A larger row is accepted only if it is a star row: its positive entries are the pairs vt , $t \in T$, of one centre v , and its negative entries are pairs R inside T . Its minimum over far-separating clusterings is at least $|T| - |R| - M$ with $M = 1$ if R contains every non-far pair inside T (then a far-free subset of T is a clique of R) and M the independence number of (T, R) otherwise, computed exactly. (If $K \subseteq T$ is the set of leaves in the cluster of v and $e_R(K)$ the number of pairs of R inside K , the left-hand side is at least $|T| - |R| - (|K| - e_R(K))$. In the first case K is a clique of R and $|K| - \binom{|K|}{2} \leq 1$; in the second, $|K| - e_R(K)$ is at most the number of connected components of (K, R) , since a component with s vertices has at least $s - 1$ edges, and hence at most the independence number of (T, R) .)
5. rounds each y_i down to a multiple of 2^{-30} and evaluates

$$\text{LB} = \sum_i b_i y_i + \sum_{p \in P} \left(\min\{0, c_p - \sum_i a_{ip} y_i\} - \min\{0, c_p\} \right) \quad (4)$$

in integer arithmetic (64-bit accumulation under a checked overflow bound, then arbitrary-precision integers), and returns $\lceil \text{LB} \rceil$;

6. optionally evaluates the cost of the archived clustering on the parsed instance, also in integers.

Rounding down keeps $y \geq 0$, so the rounded multipliers are again a dual certificate; nothing else in (4) is inexact. The sum runs over the pairs that occur in some row, since the other terms vanish. Expression (4) is (3) with $K = |N_2| = -\sum_{p \in P} \min\{0, c_p\}$ absorbed.

Proposition 6 *If the checker accepts a certificate for an instance, then the value it returns is at most OPT of that instance.*

Proof By steps 2 and 4, every row is supported on P and satisfied by the cut vector of every clustering that separates far pairs. By Lemma 1 this includes every optimal clustering, and Theorem 5(a) with the rounded multipliers gives $LB \leq OPT$. OPT is an integer. $\square$

The inequality behind the proof, that (4) is at most the cost of every far-separating clustering for any rows valid for such clusterings, is proved in Lean 4 (`far_dual_bound`, `far_dual_le_opt`; Appendix A), as is the fact that optimal clusterings separate far pairs; row validity is a hypothesis there. The procedures that decide it (steps 2–4), the reader and the integer evaluation are tested, not verified, and so is the closed form for star rows with an incomplete R , whose validity argument is the one given in step 4; no archived certificate uses that case. What remains to be trusted is therefore the checker’s reader, the enumeration and the star formula of step 4, the integer evaluation, and the Python, NumPy and Numba (LLVM) runtime; running the checker with `NUMBA_DISABLE_JIT=1` removes the compiled kernels from this list at the price of speed (Section 6.4). Every archived certificate records the edge hash computed by the solver’s loader, so the acceptance of all of them shows that the two readers agree on every instance we use. The tests of the repository check the star formula against the enumeration on random star rows with at most nine vertices, the integer cost of a clustering against the solver’s, and the rejection of certificates with a perturbed right-hand side, a negative or out-of-range multiplier, a pair outside P , a missing instance hash or a different instance. An earlier version of the checker converted the multipliers to 64-bit integers before checking their range, so a certificate with a multiplier above 2^{33} would have wrapped around to a negative value and been accepted with a wrong bound. This was found in review and fixed; the largest multiplier in any archived certificate is 14. A second review found that the enumeration summed the coefficients of a row in 64-bit integers without a bound on the row, so a row that repeats one pair thousands of times with coefficients near 2^{51} wrapped around and was accepted as valid; step 3 now rejects such rows (the largest sum of absolute coefficients of a row in any archived certificate is 210). Both cases are regression tests, and all archived certificates were re-checked with the final version (Section 6.4).

Relation to proof formats.

VIPR [18] certifies a mixed-integer program by a derivation: cutting planes and branching disjunctions, each checked syntactically. A local row of ours would need a branching derivation over all far-separating partitions of its vertices, up to 21,147 leaves per row, so we decide its validity semantically by enumerating these partitions instead, which enlarges the trusted base by the enumeration code but keeps a certificate to the rows and their multipliers. Proof logging for pseudo-Boolean and constraint solvers [20, 77] certifies combinatorial bounds of the same local kind through cutting-plane derivations, with verified checkers available; translating our rows into such a format would give a second, independent check and is left open. The certificate format is specified, independently of the checker’s code, in `docs/certificate_format.md`, including the byte layout of the edge hash and the reading rules for PACE and SNAP files, so that other solvers can write certificates; the checker accepts the packings of the KaPoCE branch-and-bound exported in this format (Section 6.3). In single-file use the parser settings are read from the certificate unless given on the command line, and are printed

as part of the result; only the directory mode of step 1 takes them from the committed instance table.

Cost of checking.

Checking is much cheaper than computing the bound: the largest certificate we report, on `exact198`, has 2,157,648 rows, and no archived certificate takes more than 253s to check, including reading the instance (Section 6.4). One command re-checks every archived certificate against freshly downloaded instance files and writes one line per certificate.

What is not certified.

The certificate proves a lower bound, and the archived clustering an upper bound; together they bound the gap of the reported solution. The approximation guarantees of Theorems 3 and 9 are statements about the algorithms, not about a particular run, and are not checked. Running times are measured, not certified.

5 Gap decomposition and CertiFlip

A dual solution does more than bound the optimum: together with a clustering it says where the clustering can still improve. This section makes that precise and uses it in a primal method that returns a certificate with every solution.

Theorem 7 *Let $y \geq 0$, $r = c - A^\top y$, and let $\mathcal{C}$ be a clustering that separates all far pairs. Then*

$$\text{cost}(\mathcal{C}) - \text{LB}(y) = \sum_{e \in P} \underbrace{(r_e x_e^{\mathcal{C}} - \min\{0, r_e\})}_{g_e \geq 0} + \sum_i y_i \underbrace{(a_i^\top x^{\mathcal{C}} - b_i)}_{s_i \geq 0}. \quad (5)$$

Proof $\text{cost}(\mathcal{C}) = K + c^\top x^{\mathcal{C}} = K + r^\top x^{\mathcal{C}} + y^\top A x^{\mathcal{C}}$. Subtract (3). Each $g_e \geq 0$ because $x_e \in \{0, 1\}$, and each $s_i \geq 0$ by the validity of row i . $\square$

Theorem 7 is complementary slackness for the pair $(x^{\mathcal{C}}, y)$, written with the box constraints, and Corollary 8 below is the reduced-cost argument behind reduced-cost fixing and persistency [54]; we state both because the LNS relies on their exact form. A pair term vanishes iff the clustering agrees with the sign of the reduced cost. A row term vanishes iff the row is tight or has multiplier 0. Assigning each term to the endpoints of its pairs gives a *gap map* over the vertices.

Corollary 8 (local certificate) *For $B \subseteq V$ let $\Gamma_B(\mathcal{C})$ be the sum of the terms of (5) whose pair, or one of whose row pairs, has an endpoint in B . Let $\mathcal{C}'$ be any clustering that separates far pairs and coincides with $\mathcal{C}$ on all pairs with both endpoints outside B . Then $\text{cost}(\mathcal{C}) - \text{cost}(\mathcal{C}') \leq \Gamma_B(\mathcal{C})$. In particular, if $\Gamma_B(\mathcal{C}) < 1$ then no re-clustering of B improves $\mathcal{C}$.*

Algorithm 1 CertiFlip(G^+, T)

```
1:  $\mathcal{C} \leftarrow \text{PIVOT}(G^+)$  with a uniformly random order
2:  $\mathcal{C} \leftarrow \text{ITERATEDFLIP}(\mathcal{C})$  ▷ insertion local search with flips
3: build the distance-two support  $P$ 
4:  $y \leftarrow$  star packing with ruin-and-recreate local search ▷ feasible dual
5:  $y \leftarrow \text{BLOCKASCENT}(P, y, \mathcal{C})$  ▷ METIS sweeps, then gap-guided blocks
6: if one block covered  $V$  and separation converged then
7:    $\mathcal{C} \leftarrow$  best of  $\mathcal{C}$  and  $\text{ITERATEDFLIP}(\text{CMSY-ROUND}(x^*))$  over several roundings
8: end if
9:  $\mathcal{C} \leftarrow \text{GAPLNS}(\mathcal{C}, y)$  until time  $T$ 
10: return  $\mathcal{C}$  and  $\lceil \text{LB}(y) \rceil$  ▷ lines 4–5 target half of the remaining budget
```

Proof Apply (5) to both clusterings. Terms not touching B coincide, and the terms of $\mathcal{C}'$ are non-negative. Costs are integers. $\square$

Gap-guided LNS.

Large-neighbourhood search [78], here with exact sub-MIPs in the spirit of RINS and local branching [79, 80], fixes the clustering outside a vertex set B ($|B| \leq 60$) and re-optimises B exactly. Let $\mathcal{K}$ be the outside clusters with a positive edge into B . Re-clusterings of B correspond to clusterings of B together with one super node per $K \in \mathcal{K}$, where super nodes must stay apart. The cost is $\sum_{uv} [uv \in E^+]x_{uv} + [uv \notin E^+](1 - x_{uv})$ for $u, v \in B$, plus $w_{vK}x_{vK} + (|K| - w_{vK})(1 - x_{vK})$ for each $v \in B$, where w_{vK} counts the positive edges between v and K . We solve this MIP with HiGHS and add triangle inequalities lazily until the solution is transitive. To keep Corollary 8 applicable, v may join K only if it is within distance two of every member of K , and far pairs inside B are fixed apart. Neighbourhoods are grown around high-gap vertices, either by breadth-first search or as a union of whole clusters. Neighbourhoods with $\Gamma_B < 1$ are skipped because they are certified optimal. We did not compare this choice with random or breadth-first neighbourhoods around arbitrary vertices, so we make no claim about its benefit; the LNS is part of CertiFlip, whose bounds and clusterings enter Section 6. Theorem 7 and Corollary 8 need a clustering that separates far pairs, which the output of the flipping search need not do. Before the search, and after every improvement, we therefore move to a singleton every vertex v with $|N(v) \cap S| < (|S| - 1)/2$ in its cluster S , until there is none. Each such move lowers the cost, and by the argument of Lemma 1 the result separates all far pairs. The certificate $\Gamma_B < 1$ covers re-clusterings of B that also separate far pairs; by Lemma 1 these include every optimal one.

Figure 3 shows the gap map at work on the neighbourhood of Figure 2. For a Pivot clustering the gap of 10 is concentrated on one cluster; the LNS re-optimises the blocks around the high-gap vertices and reaches a clustering whose cost equals the bound, which certifies it optimal.

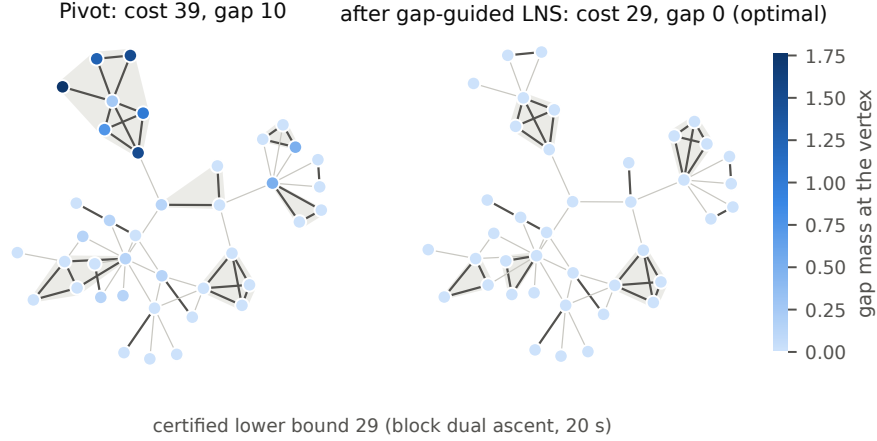


Fig. 3 Gap decomposition (Theorem 7) on the neighbourhood of Figure 2. Each vertex is coloured by its share of $\text{cost}(\mathcal{C}) - \text{LB}$ (pair terms split between the endpoints, row terms between the vertices of the row); shaded regions are clusters, dark edges lie inside a cluster. Left: a Pivot clustering. Right: after gap-guided LNS the gap is zero, so the clustering is optimal. Computed by the released code.

5.1 CertiFlip

Insertion local search.

For every root r we build one candidate insertion $S(r)$ by *cleaning* $N[r]$. Starting from $S = N[r]$, we repeat a few sweeps over $v \in N[r] \setminus \{r\}$: we put v in S iff joining S costs v less than staying in the remainder of its current cluster. Both costs are computed exactly from counts of weighted positive neighbours. We then compute the exact change of the objective in $O(\sum_{v \in S} \deg v)$ time and apply the insertion if it is negative. Rounds of insertions alternate with a multilevel (Louvain-type) vertex-move local search. That search works on the weighted objective, moves super nodes at coarse levels and hence also merges clusters. ITERATEDFLIP follows Cohen-Addad et al. [24]. Positive weights take values in $\{1, 1.5, 2\}$ (we double all weights to stay in integers). Each round flips the pairs cut by the previous solution, re-optimises, flips and re-optimises again, and combines the last three solutions with their 3-way pivot. The best candidate under the original objective is returned.

Theorem 9 *Let $\mathcal{C}$ and LB be the output of Algorithm 1.*

- (a) $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 3 \text{ OPT}$.
- (b) $\text{LB} \leq \text{OPT}$, so $\text{cost}(\mathcal{C})/\lceil \text{LB} \rceil$ is an upper bound on the approximation ratio achieved on the instance.
- (c) If line 7 is executed, $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 2.06 \text{ OPT}$.

Proof Every step after line 1 returns a clustering no worse than its input: the local searches only apply improving moves, ITERATEDFLIP returns the best of its candidates, which include the local optimum reached from its input, and GAPLNS accepts only improvements. (a) then

follows from the Pivot guarantee [3]. (b) is Theorem 5(a) and the integrality of OPT. (c) If one block covered V and separation converged, the block LP primal x^* satisfies all rows (1)–(2), so it is feasible for LP_P . A block covering V contains every row with a positive multiplier (packed rows are released into it), so its costs are the original costs c (Theorem 5(c)) and the cost of x^* is the value of the block LP. Every row of the block LP is valid for an optimal clustering (Lemma 1 and Proposition 4), whose cut vector is therefore feasible for the block LP; hence the block LP value is at most OPT. By Theorem 3(b,c), $\mathbb{E}[\text{cost}(\text{CMSY-ROUND}(x^*))] \leq 2.06 \text{ OPT}$. The remaining steps are monotone. (The LP solver returns x^* in floating point and feasible up to its tolerance; part (c) is a statement about the exact optimum, unlike part (b), which is checked exactly.) $\square$

Remark 10 Part (a) holds for any procedure that starts from a Pivot clustering and never makes its solution worse; it is a safeguard, not a distinguishing property. If the insertion local search reached an optimum with respect to *all* cluster insertions, Cohen-Addad et al. [24] would give a factor of 2, and $2 - 2/13$ with enough flipping rounds. Our candidate generation explores a polynomial subfamily of insertions, so we do not claim these factors. The certificate of Theorem 9(b), however, is valid in every case.

Running time.

Pivot takes $O(n + m)$ time. One round of insertions takes $O(\sum_v \sum_{u \in N(v)} \deg u) = O(\sum_u \deg(u)^2)$, the same order as the size of P and the number of bad triangles. The support takes $O(\sum_w \deg(w)^2)$ time and memory. The block LPs dominate the running time, and the time spent on them is set by the budget T .

Budget.

The budget T is checked between steps: before each block LP, each LNS neighbourhood and each flipping round. A single round of the insertion local search, the construction of the support and a sub-MIP are not interrupted, so a run can exceed T . Section 6 reports the measured wall-clock and CPU times of every run, including these overruns, and the time at which the final bound was reached.

6 Computational study

6.1 Setup

Implementation and machines.

All methods are implemented in Python with Numba-compiled kernels (package `ccbench`). LPs and MIPs are solved with HiGHS 1.15 [76] and graph partitions are computed with METIS [75]. The runs of CertiFlip, of the star packings, of the metric LP on P and of the head-to-head comparison were made on Google Colab virtual machines with two logical cores and 12.7 GB of memory, one run per machine at a time, with the CPU model recorded in every result file (AMD EPYC 7B12; Intel® Xeon® CPU @ 2.20GHz; the AMD model on 51 of the 223 star-packing runs, 16 of the 120 head-to-head runs at 60s, the Intel model on all others). The triangle packings, three of the KaPoCE root bounds on SNAP graphs, the rerun of its root bounds on the

PACE exact track, SCC and the rows marked ^a in Table 5 were run on 4-core workstations with several runs in parallel; these are marked where they appear. The 1,800 s bound runs and the triangle LPs on every fourth PACE exact instance (Section 6.2) were run on a laptop (12th Gen Intel[®] Core(TM) i9-12900H), and RAMA on a laptop GPU. Every run uses one thread. Budgets are wall-clock times; CPU times are recorded as well.

Instances.

Table 1 lists the SNAP graphs [81], used as complete instances with E^+ given by the edges (for signed networks, by the positive edges [82, 83]). Fifteen of them were used during development; the twelve held-out graphs were chosen afterwards to span 10^3 to 10^6 vertices and several graph classes. They were fixed before any bound was computed on them and before any run of the protocol below; seven of them are also PACE heuristic-track instances and had been run once, in an earlier comparison of the two solvers made after the code was frozen (Appendix D). We also use the 200 instances of each track of PACE 2021 [8]: the exact track ($n \leq 620$; 173 instances with known optimum and 27 open) and the heuristic track (up to $2 \cdot 10^6$ vertices). Ten heuristic-track instances coincide with development graphs and seven with held-out graphs (Table 1).

Lower bounds compared.

All bounds are rounded up to integers. Bounds marked *checked* are values returned by the checker of Section 4 from an archived certificate.

- *Triangle packings*: a maximal pair-disjoint packing of bad triangles (the bound of MatchFlipPivot [13]) and a fractional packing by the Garg–Könemann scheme with $\varepsilon = 0.05$ [32, 33], scaled by its maximum load.
- *Star local search* (SLS, checked): the star-packing local search of Section 3.2, compiled C++ (`experiments/ssstar, g++ -O2`), for 600 s on the SNAP graphs and 60 s on the PACE exact track, the budget of the CertiFlip runs there, with seeds 0–4; seed 0 is reported unless stated otherwise. It runs until the time limit, as a minimum time; the certificate is written from its stars without building the support. On `com-Youtube` and `email-EuAll` a star has at most 100 leaves. Controls: with the stopping rule of the KaPoCE branch-and-bound and 60 s only as a cap (PACE exact, seeds 0–4; the rule counts rounds, so these runs were made partly on a 4-core cloud machine with a 2.8 GHz Xeon, three at a time); 300 s on the PACE instances closed only by the CertiFlip bound; 1,200 s on the SNAP graphs with a support; and 3,600 s on the SNAP graphs on which it did not approach its stopping rule within 600 s.
- *SLS, then block LPs* (SLS+LP, checked): the star local search for 600 s, then METIS block sweeps from its packing for 600 s, on the SNAP graphs with a support.
- *Ruin-and-recreate star packing* (R&R, checked): the greedy star packing on P improved by ruin and recreate, installed as a dual solution without any LP. The number of iterations is set before the run from the iteration rate measured on the instance, so as to fill the budget; the measured running times, which can be well above it, are reported.

Table 1 SNAP instances: vertices, edges, size of the distance-two support P and role.
“_”: the estimate $m + \sum_v \binom{\deg v}{2}$ of the support exceeds $3 \cdot 10^7$ pairs, a threshold fixed in advance from the memory of the machines; CertiFlip and the block LPs were not run on these graphs, the star packings were (Table 6); *: $|P|$ as measured by the ruin-and-recreate packing, which builds the support. PACE: the heuristic-track instance with the same graph.

graph	n	m	$ P $	role	PACE
ca-GrQc	5,242	14,484	78,224	dev	heur167
ca-HepTh	9,877	25,973	205,421	dev	heur173
ca-HepPh	12,008	118,489	1,638,915	dev	heur175
ca-AstroPh	18,772	198,050	4,639,091	dev	heur176
ca-CondMat	23,133	93,439	1,169,993	dev	heur178
email-Enron	36,692	183,831	15,241,801	dev	heur179
loc-Brightkite	58,228	214,078	8,576,618	dev	
soc-Epinions	75,879	405,740	36,788,203*	dev	heur180
Slashdot+	75,144	382,882	35,019,657*	dev	
Epinions+	114,467	592,591	56,948,607*	dev	
BitcoinOTC+	5,573	18,591	972,669	dev	
BitcoinAlpha+	3,683	12,972	493,107	dev	
com-Amazon	334,863	925,872	6,933,597	dev	heur193
com-DBLP	317,080	1,049,866	13,718,957	dev	heur191
com-Youtube	1,134,890	2,987,624	–	dev	heur198
email-Eu-core	1,005	16,064	223,665	held-out	heur094
facebook	4,039	88,234	1,446,301	held-out	heur166
wiki-Vote	7,115	100,762	3,473,778	held-out	
cit-HepTh	27,770	352,285	15,155,045*	held-out	
cit-HepPh	34,546	420,877	11,577,385	held-out	
p2p-Gnutella31	62,586	147,892	1,663,263	held-out	
soc-Slashdot0902	82,168	504,230	49,698,358*	held-out	heur186
loc-Gowalla	196,591	950,327	–	held-out	heur188
email-EuAll	265,214	364,481	–	held-out	
amazon0302	262,111	899,792	6,588,959	held-out	heur189
web-NotreDame	325,729	1,090,108	–	held-out	heur192
roadNet-PA	1,088,092	1,541,898	4,421,718	held-out	heur197

- *Metric LP on P* : LP_P with triangle rows (1)–(2) only, solved by cutting planes with HiGHS until no row is violated or the budget ends; and the same with star rows. These are LP values in floating point, reported to separate the strength of the relaxation from the dual method.
- *CertiFlip bound* (checked): Theorem 5, as computed within CertiFlip, whose bound phase targets half of the remaining budget but does not interrupt a block LP (measured times in Section 6.4), and, on a subset of PACE instances, with a long budget and one block covering the graph.
- *R&R, then block LPs* (R&R+LP, checked): the ruin-and-recreate packing with half of a 1,200s budget, followed by METIS block sweeps from it for the other half, as

one procedure; by Theorem 5(b) the result is never below the packing it starts from. One run per graph (seed 0) on the SNAP graphs with a support.

- *RAMA*: the multicut dual of Abbas and Swoboda [52] (commit ee459d6, GPU solver, lower bound only) on the support P with weight $+1$ on E^+ and -1 on N_2 , plus $|N_2|$ (Section 2); run on a laptop GPU (NVIDIA RTX A2000). RAMA prints its bound with six significant digits, and the values are not certified.
- *KaPoCE root bounds*: the star-packing and P_3 -packing bounds of the KaPoCE branch-and-bound [10]. On the PACE exact track we use the values published by its authors, and check them against a rerun without data reductions (Section 6.2); these are not certified. On the SNAP graphs with at most $2.5 \cdot 10^4$ vertices we ran its lower-bound program (branch `experiments`, commit 63079a9, four-hour limit, one run per graph; three graphs on a 4-core workstation, the others on the 2-core Colab machines) ourselves; its dense matrices do not fit into memory on the larger graphs. On these graphs we also exported its final star packing and checked it as a certificate (Section 6.3).

Solvers compared.

Pivot [3], the best of 50 Pivot runs, MatchFlipPivot [13], vertex-move local search [59], our multilevel Louvain-type local search, Leiden with the constant Potts model at resolution $1/2$ [60, 62], our iterated flipping local search, CertiFlip, PXMem, and KaPoCE [7], the winner of both PACE 2021 tracks (heuristic version, commit 64e2101). SCC [9] optimises the objective of sparse signed graphs, in which a pair without an edge costs nothing, whereas here every non-edge is a negative pair. We run it (commit 05d0849, memetic version with one process) on the complete encoding, in which every non-adjacent pair is an explicit edge of weight -1 . This needs $\binom{n}{2}$ edges and is feasible on the PACE exact track (Section 6.2) but not on the SNAP graphs.

6.2 Lower bounds on the PACE exact track

Same heuristic, different stopping rules.

The published root bound of the KaPoCE branch-and-bound and our star local search pack the same induced stars with the same moves; they differ in the implementation and in when they stop. The published data state neither the time spent on the root bounds nor whether they were computed after the data reductions of the branch-and-bound, so we reran its lower-bound program on the input graphs, without reductions (the program of Section 6.3, 900s limit, one run per instance; 62 on the Colab machines, one at a time, the others on a 4-core cloud machine with a 2.2 GHz Xeon, at most three at a time). The star packing finished within the limit on 199 of the 200 instances (`exact198` stopped after the P_3 packing). Its P_3 packing equals the published value on 200 of 200, its star packing on 164, and on the others the star packing differs by at most 5 (20 larger, 15 smaller, at most 1.06%; the heuristic uses a random generator, which we ran with its default seed, and the published values may come from another version). The published root bounds are therefore bounds of the input graphs. The heuristic stops by its own rule, once the rounds are more than five times the rounds that improved the packing; its star bound took a median of 0.7 s (at most 522 s).

Table 2 PACE 2021 exact track, the 173 instances with known optimum: lower bounds. Checked (values returned by the checker from archived certificates): the ruin-and-recreate star packing, the CertiFlip bound (budget 60 s), the CertiFlip bound with 1,800 s (every fourth instance), the star local search run until 60 s, the star local search with the stopping rule of the branch-and-bound (60 s cap), and the larger of the CertiFlip bound and the star local search. Not certified: the triangle packings, computed earlier on a 4-core workstation; the metric LP on P , a floating-point LP value reported only where cutting planes converged within 900 s (every fourth instance); the root bounds of the KaPoCE branch-and-bound as published by its authors, and its star packing as rerun by us without data reductions; and the last row, which takes the larger of a checked and a published value. Seed 0 throughout. time: median measured time of the runs behind the row, including the certificate (for the CertiFlip bound, the whole CertiFlip run; for the rerun, its star bound); –: not measured or not published.

bound	instances	mean LB/OPT	min LB/OPT	LB = OPT	time (s)
greedy triangle packing	173	0.8612	0.660	14	–
fractional triangle packing	173	0.9664	0.811	30	–
ruin-and-recreate star packing	173	0.8712	0.247	17	69.1
CertiFlip bound	173	0.9874	0.816	106	18.6
CertiFlip bound, 1800 s	45	0.9948	0.940	34	676.5
star local search (SLS)	173	0.9984	0.960	122	60.1
SLS, B&B stopping rule	173	0.9949	0.929	83	0.5
metric LP on P , triangle rows	39	0.9762	0.814	21	1.0
B&B star packing [10]	173	0.9948	0.891	79	–
B&B P_3 packing [10]	173	0.9742	0.813	59	–
B&B star packing, our rerun	173	0.9950	0.891	81	0.4
larger of CertiFlip and SLS	173	0.9989	0.972	130	–
larger of both and B&B star	173	0.9989	0.972	131	–

Table 3 Mean LB/OPT on the solved PACE exact instances by edge density $m/\binom{n}{2}$. The fractional triangle packing was computed earlier on a 4-core workstation.

edge density	instances	CertiFlip bound	star local search	ruin-and-recreate star packing	fractional triangle packing	B&B star packing
[0, 0.1)	35	0.9978	0.9977	0.9529	0.9614	0.9934
[0.1, 0.3)	40	0.9781	0.9990	0.8623	0.9646	0.9971
[0.3, 0.5)	48	0.9823	0.9973	0.9046	0.9448	0.9914
[0.5, 1)	50	0.9923	0.9994	0.7892	0.9922	0.9974

Under the same rule, with 60 s only as a cap, the star local search proves optimality on 83 of the 173 solved instances (mean LB/OPT 0.9949), against 79 for the published root bound (mean 0.9948); it is larger than that bound on 43 instances, equal on 85 and smaller on 45, and took a median of 0.5 s including its certificate (Table 2); 5 runs reached the cap. Over five seeds its count ranges from 81 to 85. The reimplementations are thus equivalent to the original where both run, which is what it was written to be. Run until its 60 s time limit instead, the same search proves optimality on 122 instances (mean 0.9984; from 122 to 124 over five seeds, mean 0.9984 to 0.9985), and at least 0.960 of the optimum on every instance. With seed 1, on the 43 instances that the 60 s run closes and the run with the stopping rule does not, the 60 s run reached the optimum after a median of 3.9 s (at most 60 s). The published root bounds are therefore far from what the same heuristic reaches with a few seconds more; the dense implementation could be run longer on these instances as well, and our comparison says

nothing about which implementation is faster per round. Our earlier ruin-and-recreate packing reaches only 0.8712 on average.

Block LPs.

The CertiFlip bound, a ruin-and-recreate packing followed by block LPs within a 60 s CertiFlip run, proves optimality on 106 instances and is larger than the star local search on 9, by one to eight units. On 8 of these it is optimal and the star local search (seed 0, 60 s) is not. Neither four further seeds nor a run of 300 s close 7 of them (closed: `exact003`), and their densities range from 0.01 to 0.38: there the block LPs close units that more packing time does not. Together, the two checked bounds prove optimality on 130 instances (mean 0.9989), at a cost of two runs of 60 s nominal each; adding the published root bound to them proves optimality on only one more (131). Below density 0.1 the two checked bounds are equal on average (Table 3).

On every fourth instance we also ran the bound with 1,800 s and one block covering the graph, and the metric LP on P with triangle rows only. With the longer budget the mean LB/OPT on these 45 instances is 0.9948, against 0.9858 with the 60 s runs and 0.9966 for the B&B star packing, and the bound is optimal on 34 of them (28 at 60 s). The triangle LP converged within 900 s on 39 of these instances, with a mean LB/OPT of 0.9762 (0.9911 for our bound on the same instances). Of the 50 bound runs, 49 were made on a laptop (12th Gen Intel® Core(TM) i9-12900H), several at a time, and one on a Colab machine. The limit applies to the LP phase; the star-packing warm start before it (sized to about 30% of the limit) and the certificate come on top, and 13 of the 50 runs took longer than 1,800 s, the longest 5227 s. The checker accepted all their certificates.

Open instances.

Of the 27 instances of the exact track without a known optimum, our best checked bound exceeds the larger published root bound [10] on 22 (Table 4); on `exact179` it leaves a gap of one to the best known solution. On the other 5 it is smaller than the published root bound, by at most 0.7%. None of the open instances is closed.

Primal quality.

On the exact track the primal side is not the bottleneck (Table 5). CertiFlip finds the best known value on 192 of the 200 instances although it spends a large part of its 60 s on the bound; SCC on the complete encoding and KaPoCE find it on all of them. The certified gaps on this track are therefore determined by the bound.

6.3 Certified gaps on SNAP graphs

Certified gaps.

Every one of the 27 graphs now has a checked lower bound (Table 6). On the 23 graphs with an archived clustering, the largest checked bound certifies it within 0.22% to 20.9% (median 3.1%); on the other 4 (`soc-Epinions`, `Slashdot+`, `Epinions+`, `com-Youtube`) only gap*, against the best value of any run, is available. The triangle packings, the lower bounds used at this scale so far, leave gaps of 17% to 90%

Table 4 Open instances of the PACE 2021 exact track on which the checked lower bound improves the published one. Published bounds: the best upper bound and the larger of the two root bounds of [10]; our UB: the CertiFlip run of Table 2; checked LB: the largest checked bound of our runs.

instance	n	m	published UB	published LB	our UB	checked LB
exact042	100	1563	1,055	1,006	1,055	1,017
exact043	100	1624	984	968	984	978
exact051	120	2251	1,619	1,508	1,619	1,533
exact052	120	2448	1,473	1,431	1,473	1,442
exact060	124	2288	1,491	1,427	1,491	1,446
exact069	140	2915	2,140	1,994	2,140	2,015
exact070	140	3014	1,792	1,762	1,792	1,779
exact082	160	4015	2,434	2,397	2,434	2,414
exact083	160	4259	2,939	2,808	2,941	2,824
exact091	180	4602	3,234	3,072	3,236	3,087
exact092	180	6067	3,563	3,520	3,563	3,529
exact101	200	6731	4,198	4,135	4,198	4,148
exact102	200	6746	3,925	3,895	3,925	3,903
exact103	200	7081	4,572	4,381	4,572	4,382
exact105	200	12309	5,320	5,314	5,320	5,318
exact179	330	1370	633	620	633	632
exact180	330	2256	1,081	1,052	1,083	1,073
exact181	330	3253	1,563	1,513	1,568	1,532
exact182	330	4329	2,159	2,091	2,165	2,100
exact183	330	5613	2,789	2,717	2,788	2,734
exact184	330	7270	3,684	3,582	3,692	3,597
exact195	620	3992	2,529	2,450	2,533	2,476

Table 5 PACE 2021 exact track (200 instances): primal quality against the best known value (published upper bound or better value of any run). CertiFlip: the certificate runs of Table 2, one seed on the Colab machines; SCC: one seed on the complete encoding, on a 4-core machine with three runs in parallel; ^a: best of three seeds from earlier runs on the same 4-core machine with four runs in parallel.

method	best known	mean gap (%)	max gap (%)
CertiFlip (60 s)	192/200	0.006	0.32
SCC [9] (60 s)	200/200	0.000	0.00
KaPoCE (60 s) ^a	200/200	0.000	0.00
IteratedFlip ^a	187/200	0.007	0.32
Leiden-CPM ^a	158/200	0.119	3.00
Pivot ^a	13/200	32.635	229.54

(median 46%) on the graphs with a support. The median gap is 1.0% on the development graphs and 4.0% on the held-out graphs. The split is descriptive: the bound procedures were not evaluated out of sample (Section 1), and the held-out set contains the denser social and citation networks and four of the five largest graphs. The largest gaps are on **web-NotreDame** (20.9%), a web graph with hubs of more than 10^4 links, and on the held-out voting, citation and e-mail graphs **wiki-Vote**, **cit-HepPh** and **email-EuAll**; the column gap* shows that no solver we ran closes them from above, so we cannot tell how they divide between the two bounds.

Table 6 SNAP graphs: checked lower bounds and certified gaps. UB: the best archived clustering (re-checked from its file; – if none was archived). Checked LB, one column per procedure: CertiFlip, the best of three runs of 600 s (seeds 0–2; only on graphs whose support is below the threshold of Table 1); R&R packing, the ruin-and-recreate star packing alone; R&R+LP, that packing followed by block LPs (1,200 s); SLS, the star local search of Section 3.2 (600 s; k : at most 100 leaves per star); SLS+LP, the star local search followed by block LPs (600 s each); one run each with seed 0; the largest is in bold. B&B root: the root star-packing bound of the KaPoCE branch-and-bound (up to four hours), run on the graphs with at most $2.5 \cdot 10^4$ vertices; its packing was exported and accepted by our checker ($^\circ$ where not), † where it exceeds all our checked bounds; t.o.: no bound within four hours; mem.: out of memory. gap: $(UB - LB)/LB$ with the largest checked bound of any archived certificate for the graph, including the runs of Table 8 (§ : from the packing run of Table B2; ‡ : from a run of Table 8); gap*: the same with the best value of any run of any solver (reported, not archived); tri. factor: best known value divided by the better of the two triangle packings. Upper block: development graphs; lower block: held-out graphs.

graph	UB	checked LB					B&B root	gap (%)	gap* (%)	tri. factor
		CertiFlip	R&R packing	R&R +LP	SLS	SLS +LP				
ca-GrQc	6,051	6,036	6,003	6,034	6,029	6,038	6,015	0.22	0.20	1.28
ca-HepTh	15,861	15,682	15,705	15,725	15,697	15,706	15,670	0.83 §	0.53	1.45
ca-HepPh	46,194	43,199	42,865	43,740	45,969	45,974	46,014 †	0.42 ‡	0.29	1.17
ca-AstroPh	125,682	118,224	117,865	119,101	123,935	123,955	t.o.	1.05 ‡	0.93	1.46
ca-CondMat	56,974	55,283	55,627	55,910	56,420	56,417	mem.	0.88 ‡	0.63	1.43
email-Enron	150,701	140,551	140,854	140,862	144,207	144,166	–	4.07 ‡	3.88	1.77
loc-Brightkite	175,081	163,366	163,927	164,106	169,694	169,661	–	2.65 ‡	2.43	1.71
soc-Epinions	–	–	344,995	–	349,706	–	–	–	4.07	–
Slashdot+	–	–	334,298	–	336,751	–	–	–	3.78	–
Epinions+	–	–	489,582	–	498,929	–	–	–	4.24	–
BitcoinOTC+	16,497	15,582	15,642	15,661	15,909	15,911	15,927	3.51 ‡	3.29	1.80
BitcoinAlpha+	11,461	10,749	10,802	10,820	11,061	11,062	11,038	3.36 ‡	3.15	1.81
com-Amazon	615,035	562,505	574,956	576,376	594,310	593,290	–	3.02 ‡	2.57	1.43
com-DBLP	608,183	582,821	589,979	589,275	600,982	600,373	–	0.92 ‡	0.68	1.42
com-Youtube	–	–	–	–	2,516,213 k	–	–	–	5.29	–
email-Eu-core	12,732	11,657	11,413	11,651	12,251	12,227	12,228	3.66 ‡	3.66	1.59
facebook	53,547	43,344	42,953	44,025	52,330	52,347	52,739 †	1.96 ‡	1.96	1.37
wiki-Vote	95,263	85,487	85,522	85,548	87,595	87,600	t.o.	8.57 ‡	8.57	1.90
cit-HepTh	293,167	–	266,019	–	277,208	–	–	4.70 ‡	4.70	–
cit-HepPh	357,463	322,666	324,327	324,608	336,163	336,120	–	5.19 ‡	5.18	1.72
p2p-Gnutella31	131,362	130,297	130,336	130,326	130,020	130,010	–	0.63 §	0.63	1.79
soc-Slashdot0902	470,778	–	447,216	–	449,403	–	–	3.89 ‡	3.88	–
loc-Gowalla	788,550	–	–	–	750,395	–	–	4.13 ‡	4.12	–
email-EuAll	342,709	–	–	–	321,593 k	–	–	4.08 ‡	4.08	–
amazon0302	540,844	503,032	511,221	508,994	531,655	531,166	–	1.26 ‡	1.26	1.32
web-NotreDame	733,709	–	–	–	576,001	–	–	20.85 ‡	20.85	–
roadNet-PA	944,542	865,131	884,166	905,670	891,590	915,741	–	3.15	3.14	1.30

The star local search.

With seed 0 and 600 s, the star local search, alone or followed by block LPs, gives a larger bound than all of our earlier procedures on 21 graphs and a smaller one on 2 (ca-HepTh, p2p-Gnutella31, where a 1,200 s ruin-and-recreate packing is larger; Table B2), and the only bound on 4 (com-Youtube, loc-Gowalla, email-EuAll, web-NotreDame), where the earlier implementation of the packing materialised the support and ran out of memory. The largest change is on **facebook**, whose gap falls from 21.6% to 2.0%; over the 20 graphs that had a checked bound and an archived

Table 7 The metric LP restricted to P , solved by cutting planes with HiGHS on one core (budget 1,800s with triangle rows and 3,600s with star rows, 7,200s on **ca-GrQc**), against the best checked bound of Table 6, and the multicut dual bound of RAMA on the same support plus $|N_2|$ (Section 2; default parameters, on one GPU; failed: the solver stopped with a CUDA illegal-address error; a value ≤ 0 is the trivial bound up to the printing precision). LP and RAMA values are in floating point and not certified. A value in parentheses is that of the last LP when the budget ended during its solution; it is not the optimum of any relaxation and not a bound. killed: the job’s process died three times without a result, most likely for lack of memory; n.r.: not run.

graph	best	best checked	LP $_P$, triangle rows		LP $_P$, + star rows		RAMA	
	known	LB	value	time (s)	value	time (s)	value	time (s)
ca-GrQc	6,050	6,038	4,931	2	6,039	2578	3,944	30
ca-HepTh	15,813	15,730	11,477	6	(14,449)	3621	8,625	2
ca-HepPh	46,133	46,001	(36,932)	1808	n.r.		−4	3
ca-AstroPh	125,541	124,381	(73,161)	1821	n.r.		failed	
ca-CondMat	56,834	56,478	41,820	797	n.r.		24,394	319
email-Enron	150,419	144,804	killed		n.r.		−30	11
loc-Brightkite	174,695	170,553	killed		n.r.		failed	
BitcoinOTC+	16,462	15,937	9,262	34	(11,312)	3605	0	4
BitcoinAlpha+	11,437	11,088	6,475	19	(8,434)	3604	2,737	444
com-Amazon	612,307	596,984	killed		n.r.		362,915	257
com-DBLP	606,723	602,651	killed		n.r.		failed	
email-Eu-core	12,732	12,282	8,032	207	(8,523)	3618	failed	
facebook	53,547	52,518	(21,639)	1805	(43,700)	3610	−3	147
wiki-Vote	95,259	87,743	killed		n.r.		−4	4
cit-HepPh	357,413	339,821	killed		n.r.		8	747
p2p-Gnutella31	131,362	130,546	73,941	67	n.r.		71,311	3
amazon0302	540,839	534,122	(429,905)	1825	n.r.		326,517	144
roadNet-PA	944,519	915,741	769,127	831	n.r.		730,910	5

clustering before, the median gap falls from 5.6% to 2.8%. A run took a median of 606s and at most 641s, including the certificate: the time limit is checked between stars, and the stars are written as certificate rows directly, without building the support. On **com-Youtube** and **email-EuAll**, whose largest degrees are $2.9 \cdot 10^4$ and $7.6 \cdot 10^3$, the unrestricted search was killed three times for lack of memory (a star with k leaves occupies $k + \binom{k}{2}$ pairs); with at most 100 leaves per star it used about 4GB and finished.

Seeds and convergence.

The search stops on wall-clock time, so its result depends on the seed and on the speed of the machine (Table 8). Over five seeds the range of its 600s bound is a median of 0.14% of its median and at most 1.59% (**web-NotreDame**). Differences between single runs of this size carry no information about the methods. On 17 of the 27 graphs every round of the 600s run improved the packing, so the search was far from its stopping rule; on the largest graphs it completed only a few rounds. Run for 3,600s on the graphs with fewer than 100 rounds, all of them improving (16 runs finished), the bound rose by a median of 0.49% and by up to 5.4% (**web-NotreDame**; Figure 4); on **com-Youtube** the longer run was killed three times, most likely for lack of memory, as its packing grows. The gaps on these graphs are therefore partly a matter of budget. The largest checked bound of a graph, which enters the column gap of Table 6, is

Table 8 The star local search on the SNAP graphs (checked values). rounds (impr.): rounds of the 600 s run with seed 0 and the rounds among them that improved the packing; seed 0: its bound; spread: the range of the bound over seeds 0–4 at 600 s, relative to its median; 1,200 s: one run of 1,200 s (seed 0) on the graphs with a support; SLS+LP: 600 s of search followed by 600 s of block LPs, as in Table 6; 3,600 s and rounds: one run of 3,600 s on the graphs with fewer than 100 rounds at 600 s, all improving; gap: $(UB - LB)/LB$ of the archived clustering against the seed-0 run alone.

graph	rounds (impr.)	seed 0	spread %	1,200 s	SLS+LP	3,600 s	rounds	gap %
ca-GrQc	7245 (56)	6,029	0.03	6,031	6,038	–	–	0.36
ca-HepTh	2588 (175)	15,697	0.09	15,711	15,706	–	–	1.04
ca-HepPh	155 (91)	45,969	0.05	46,001	45,974	–	–	0.49
ca-AstroPh	87 (87)	123,935	0.09	124,167	123,955	124,381	420	1.41
ca-CondMat	507 (241)	56,420	0.05	56,478	56,417	–	–	0.98
email-Enron	38 (38)	144,207	0.14	144,531	144,166	144,804	203	4.50
loc-Brightkite	62 (62)	169,694	0.18	170,245	169,661	170,553	288	3.17
soc-Epinions	9 (9)	349,706	0.14	–	–	350,862	63	–
Slashdot+	14 (14)	336,751	0.21	–	–	339,024	55	–
Epinions+	5 (5)	498,929	0.07	–	–	500,570	29	–
BitcoinOTC+	541 (140)	15,909	0.09	15,937	15,911	–	–	3.70
BitcoinAlpha+	1104 (142)	11,061	0.09	11,088	11,062	–	–	3.62
com-Amazon	84 (84)	594,310	0.24	594,919	593,290	596,984	351	3.49
com-DBLP	51 (51)	600,982	0.13	601,440	600,373	602,651	208	1.20
com-Youtube	1 (1)	2,516,213	0.27	–	–	–	–	–
email-Eu-core	1942 (183)	12,251	0.11	12,282	12,227	–	–	3.93
facebook	152 (136)	52,330	0.18	52,518	52,347	–	–	2.33
wiki-Vote	62 (61)	87,595	0.10	87,743	87,600	–	–	8.75
cit-HepTh	20 (20)	277,208	0.14	–	–	280,009	148	5.76
cit-HepPh	26 (26)	336,163	0.34	337,815	336,120	339,821	208	6.34
p2p-Gnutella31	207 (132)	130,020	0.04	130,068	130,010	–	–	1.03
soc-Slashdot0902	6 (6)	449,403	0.21	–	–	453,147	43	4.76
loc-Gowalla	3 (3)	750,395	0.53	–	–	757,280	13	5.08
email-EuAll	7 (7)	321,593	0.59	–	–	329,275	25	6.57
amazon0302	71 (71)	531,655	0.19	532,703	531,166	534,122	304	1.73
web-NotreDame	2 (2)	576,001	1.59	–	–	607,100	10	27.38
roadNet-PA	76 (76)	891,590	0.08	893,013	915,741	894,299	397	5.94

the maximum over all archived certificates for it, these longer runs and further seeds included; the last column of Table 8 gives the gap of the single 600 s run.

Block LPs after the star local search.

Starting the block LPs from the packing of the star local search (column SLS+LP, half of the 1,200 s for each phase) raised the checked bound above that packing on 15 of the 18 graphs with a support, by more than 0.005% on 8 of them, by a median of 0.00% and by at most 2.7% on **roadNet-PA**, whose road network has many short cycles and few large induced stars. This gain does not pay for the time: against the star local search run for the whole 1,200 s, SLS+LP is larger on only 2 of the 18 graphs (**ca-GrQc**, **roadNet-PA**) and smaller on 16; the difference has a median of -0.17% and ranges from -0.5% (**cit-HepPh**) to 2.5% (**roadNet-PA**). On the social, collaboration, citation and web graphs the time is better spent on the packing; the block LPs pay only where short cycles, not stars, carry the bound. With the ruin-and-recreate packing as the start the block LPs had added a median of 0.14% (column R&R+LP), because that packing leaves more room; the comparison of the two uses of that budget with the ruin-and-recreate packing is in Appendix B.

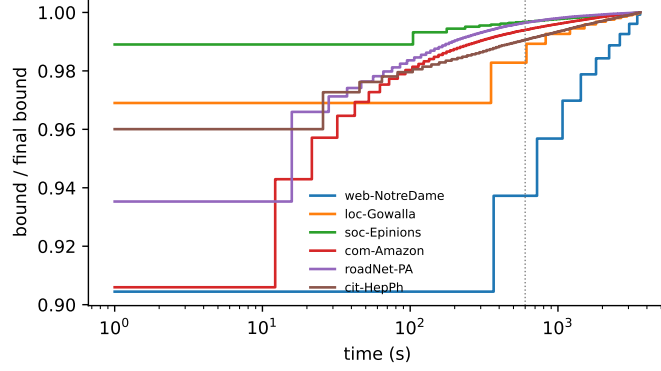


Fig. 4 The bound of the star local search over time in the 3,600 s runs, relative to its final checked value (intermediate values as reported by the search, logarithmic time axis; dotted line: 600 s).

The root bound of the branch-and-bound.

On the graphs with at most $2.5 \cdot 10^4$ vertices, the root star-packing bound of the KaPoCE branch-and-bound finished within four hours on 7 graphs (Table 6). It exceeds all of our checked bounds on 2 of them (*ca-HepPh*, *facebook*), by 0.0% to 0.4%, and is below them on the others (*ca-GrQc*, *ca-HepTh*, *BitcoinOTC+*, *BitcoinAlpha+*, *email-Eu-core*), by at most 0.45%. Its star bound took 46 s to 2.7 h, against at most 641 s for the star local search. On *ca-AstroPh*, *wiki-Vote* it did not finish within the four hours, and on *ca-CondMat* it needs about 11.6 GB and was killed in each of four runs on the machines with 12.7 GB, the longest after 3.8 h; its dense matrices, of which it keeps several with n^2 entries, do not fit into memory above $2.5 \cdot 10^4$ vertices. The two implement the same heuristic. On the graphs where the root bound is larger, its margin exceeds the spread of our search over seeds (Table 8), and our longest runs do not close it; we have not analysed whether this comes from the four-hour budget or from differences in the candidate generation of the two implementations. Its packings use the same rows as ours, so we exported them from its code (a function of 13 lines that writes the stars of its final packing, centre first; `experiments/kapoce/`), converted each into a dual solution on P and ran our checker on it. The checker accepted all 7, each with the value KaPoCE reports: the same certificate format and checker certify the bounds of another solver.

Strength of the relaxation.

Table 7 separates the strength of the relaxation from the way it is solved. On *ca-GrQc* the metric LP on P with triangle rows converges in 2 s to 4,931, 18.5% below the best known value (which is 22.7% above it). With star rows the LP reaches 6,039, 0.18% below the best known value and slightly above the best checked bound (6,038), after 2578 s. The triangle inequalities, which is what the classical metric LP has, are therefore far from sufficient on this graph; the star inequalities close almost all of the gap, and the anytime dual reaches nearly the same value in a tenth of the time. RAMA, a dual method for the multicut LP with cycle inequalities, run with its default

parameters, stays below LP_P with triangle rows on every graph where that converged, consistent with the argument of Section 2. Its values are not converged, however: they are below the better triangle packing, a feasible dual of the same cycle LP, on 13 of the 14 graphs where it ran. Its bound is at most 0.80 of the best checked bound (on `roadNet-PA`); on 6 graphs (`ca-HepPh`, `email-Enron`, `BitcoinOTC+`, `facebook`, `wiki-Vote`, `cit-HepPh`) it returned the trivial bound, zero up to its printing precision (six significant digits, which is up to ± 50 on `email-Enron`); and it stopped with a CUDA illegal-address error on 4 graphs.

6.4 Running times and checking

The budgets are not hard limits (Section 5, paragraph Budget). Of the 200 CertiFlip runs on the PACE exact track with $T = 60$ s, 41 took more than $1.01T$; the median wall-clock time was 25 s and the maximum 393 s. Of the 54 runs on SNAP graphs with $T = 600$ s, 14 took more than $1.01T$ (median 602 s, maximum 1000 s). Of these, 3 PACE runs and 3 SNAP runs exceeded $1.2T$. The bound phase of CertiFlip targets half of the remaining budget but does not interrupt a block LP. On the PACE exact track it took a median of 20 s and more than 30 s on 85 of the 200 runs (maximum 84 s); on the SNAP graphs a median of 290 s and more than 300 s on 17 of 54 runs (maximum 946 s, on `email-Enron`). The LP-rounding start (line 7 of Algorithm 1) was executed on 118 of the PACE runs and improved the clustering on 2 of them; it was never executed on the SNAP graphs. The large overruns come from steps that are not interrupted: single block LPs on `email-Enron`, and on the three PACE instances with 620 vertices and $9 \cdot 10^4$ to $2 \cdot 10^5$ edges the bound phase and, on two of them, the LP-rounding start with the local search after it (up to 277 s on `exact199`). The star local search checks its time limit between stars and took a median of 606 s (at most 641 s) on the SNAP graphs and 60 s (at most 67 s) on the PACE exact track, including its certificate. The ruin-and-recreate packing fixes its number of iterations in advance from a measured rate; on the SNAP graphs (nominal 600 s) it took 47 to 1285 s, and on the PACE exact track (nominal 60 s) a median of 68 s, more than 60 s on 150 of 200 runs and at most 1588 s. The long packing followed by block LPs (nominal 1,200 s) took a median of 780 s and more than 1,260 s on 2 graphs (at most 2188 s). All times are in the archived result files. The comparisons of different bound procedures in Sections 6.2 and 6.3 are therefore at comparable, not equal, running times, except for the controls of the star local search, which compare it under the same stopping rule or the same budget; the head-to-head comparison of Appendix C.5 is at equal elapsed time by construction.

The checker accepted all 200 certificates of the PACE runs, all 54 of the CertiFlip runs on SNAP graphs and all 223 of the star-packing runs when they were written, and rejected none. After the review of the checker (Section 4) we re-checked all 1770 archived certificates, including those of the runs made after the review, and 24 archived clusterings with its final version (commit `cb854f4`) against freshly downloaded instance files whose hashes are looked up in the committed manifest and whose parser settings are taken from the committed instance table: 1794 of the 1794 were accepted, every accepted bound equals the one checked during its run, the slowest took 253 s and all together 0.9 h (`results/mpc/recheck.csv` and `recheck_local.csv`).

The largest certificate, on **exact198**, has 2,157,648 rows. No accepted certificate uses the independence-number case of the star formula (0 rows), and the largest multiplier is 14. Run without its compiled kernels (`NUMBA_DISABLE_JIT=1`), the checker accepted 62 certificates, every fifth star local search and every tenth CertiFlip certificate of the PACE exact track and two on **ca-GrQc**, with the same values.

7 Conclusion

Lower bounds that scale to large correlation clustering instances, packings of bad triangles, certify only factors of 1.17 to 1.90 on our graphs. The stronger star packings of exact solvers were limited by a dense implementation. Because every star packing, and the metric LP with its strengthenings, can be restricted to the pairs at distance at most two, the star-packing heuristic of the KaPoCE branch-and-bound runs with memory linear in the graph and the packing, and a dual block-coordinate ascent turns the restricted relaxation into an anytime bound that starts from its packing. This gives checked bounds on all SNAP graphs of our study, up to 10^6 vertices. The bounds are exported as certificates that an independent checker verifies against the raw instance in exact arithmetic; the same checker accepts the packings of the branch-and-bound.

On the PACE exact track the reimplementations matches the published root bounds under the branch-and-bound’s own stopping rule. The larger number of optimality proofs at 60s comes from running the same heuristic longer, not from a better heuristic; the block LPs add a few instances on which more packing time does not help.

Limitations.

- The star local search implements the heuristic of an existing solver; what is new is that it runs on the support, with its certificate, at a scale where the dense implementation does not. Where both finish, on seven SNAP graphs, the root bound of the branch-and-bound, run for up to four hours, exceeds ours on 2 of them by at most 0.4%.
- The search stops on wall-clock time, so a seed does not fix its result; over five seeds its bound varies by up to 1.59% on the SNAP graphs. On the largest graphs it completes only a few rounds in 600s, and with 3,600s the bound rises by up to 5.4%; the gaps on these graphs are partly a matter of budget.
- At an equal budget the block LPs beat the star local search alone on only 2 of the 18 SNAP graphs with a support (**ca-GrQc**, **roadNet-PA**); on the PACE exact track they close 7 instances that five times the packing time does not. Elsewhere the time is better spent on the packing. We did not optimise how to split a budget between the two phases, and we did not try first-order LP methods, whose duals the checker would accept as well.
- The remaining gaps on the social, citation and web graphs (up to 20.9%) are not closed from either side, and we cannot tell how they divide between the lower and the upper bound.
- A star with k leaves occupies $k + \binom{k}{2}$ pairs. On the two graphs with the largest hubs we capped the stars at 100 leaves to fit into 12.7 GB.

- We consider unweighted instances only, and compute the bounds on the input graph. Twin contraction and other exact reductions are used on the primal side only; bounds on the reduced, weighted instance would need the distance-two rule in a weighted form and a checker that verifies the reduction.
- The checker is short but not formally verified; the formal development covers the inequality it evaluates, not the procedures that decide the validity of a row (Appendix A).
- The gap-guided choice of LNS neighbourhoods is not evaluated separately. The a-priori guarantees of our primal methods are inherited from Pivot, and the 2.06 guarantee applies only when one block covers the graph and separation converged.

Open directions.

Using the stronger root bound inside the branch-and-bound; lower bounds on reduced instances; $K_{a,b}$ packings, which Bläsius et al. [10] found comparable to stars; the split of the budget between packing and block LPs; first-order and parallel block solves; cluster-LP columns inside blocks, to which the roundings of Cao et al. [5] and García-Soriano and Schohn [6] could be applied; and a verified checker, for example by extracting it from the Lean development or by translating the rows into a proof format with a verified checker.

Supplementary information. The code, the instance lists, the raw results of every run and the archived certificates are available at https://github.com/vinhqdang/correlation_clustering; the file REPRODUCE.md lists one command per table and figure.

Declarations

- Funding: not applicable.
- Competing interests: the author declares no competing interests.
- Data availability: all instances are public (PACE 2021 and SNAP); the repository contains scripts that download them and check their SHA-256 hashes.
- Code availability: see the supplementary information.

Appendix A Formal development

The correctness of the solvers rests on short combinatorial arguments and on exact formulas that the implementation evaluates millions of times per second. An error in a sign or a boundary case would not crash the code; it would silently produce worse or invalid results. We formalised part of the mathematics in Lean 4 [84] with Mathlib [85]. The development (`lean/CCProofs`, about 1,800 lines in fourteen files) is checked by the Lean kernel without `sorry` and depends only on the axioms `propext`, `Classical.choice` and `Quot.sound`. Table A1 lists what it covers. It does *not* cover Theorem 3, the block step and the convergence statement of Theorem 5(b,c), the gap decomposition (Theorem 7) or its local corollary, the 2.06 part of Theorem 9, or any code. The Pivot guarantee enters as a hypothesis.

The formal model is that of Section 2: a finite vertex set, a simple graph G^+ and a labelling $c : V \rightarrow \alpha$ into a type with decidable equality. The cost is summed

over ordered pairs, so every count appears multiplied by two (`cost_eq_two_mul_card` relates it to the unordered count). The weighted instance has integer node sizes and symmetric weights, as in (C1).

The formal statements connect to the code in two places. The bounds we report are values of the expression that `far_dual_bound` bounds, evaluated by the checker of Section 4. The move, swap and contraction formulas are tested against recomputation: exact costs of random clusterings on contracted instances, and the tracked cost against a full recomputation at the end of annealing runs with many moves, swaps and rollbacks. These tests check final states, not every step.

Appendix B The ruin-and-recreate packing and the block LPs

This appendix reports the runs that separated the effect of the packing from that of the block LPs before the star local search was available; they use the ruin-and-recreate packing of Section 3.2 (columns R&R and R&R+LP of Table 6).

A long packing, then block LPs.

Running the ruin-and-recreate packing first with the iterations of the separate packing run (nominally half of a 1,200s budget; it took a median of 200s) and the block LPs from it for the other half (in total a median of 780s, at most 2188s; column R&R+LP) separates the two effects. The block LPs raised the bound above the packing they started from on all 18 of the 18 graphs, by a median of 0.14% and by up to 2.8% on `roadNet-PA`, and the combined procedure gives the largest checked bound on 13 of the 18 graphs, 7 of them by less than 0.2%. This is one seed against the best of three CertiFlip seeds, and the procedure ran longer: a median of 780s against 289s for the bound phase of CertiFlip. On the other 5 it is below the best of the other runs by at most 2,227: on `ca-GrQc` and `email-Eu-core` below CertiFlip by 2 and 6, and on `com-DBLP`, `p2p-Gnutella31` and `amazon0302` below the separate packing run, because its own packing phase ended lower: the local search is deterministic for a seed, but its number of iterations is set from a rate measured at the start of each run, and the two runs differ in it (the counts are not logged). With a strong packing as the start, the block LPs therefore add to the bound on every graph; on the large sparse graphs what the CertiFlip bound lacks is time for the packing, not the LP part.

The packing with the same budget.

Whether the LP phase is worth its time is a different question: the other half of the budget could also go to the packing. We therefore ran the packing alone with the whole 1,200s budget (seed 0, Table B2). Its iteration count is fixed from the measured rate as before, and the run took a median of 302s, at most 2043s, so the budgets are equal only nominally, in favour of the packing when it overruns and of the block LPs otherwise. The combination gives the larger bound on 10 of the 18 graphs, by up to 2.2%, among them five of the six graphs with $m/n > 9$ and `roadNet-PA`; the longer packing is larger on 8 (`ca-HepTh`, `email-Enron`, `loc-Brightkite`, `com-Amazon`, `com-DBLP`, `cit-HepPh`, `p2p-Gnutella31`, `amazon0302`), by up to 1.3%, and on all of

Table A1 Coverage of the formal development (Lean 4, Mathlib). “Full”: the statement of the paper is proved as stated (up to the ordered-pair convention); “partial”: the part named is proved; “reduction”: proved from the cited guarantee taken as a hypothesis; “_”: not formalised.

result	coverage	Lean theorems
Lemma 1	partial	far pairs are separated by every optimal clustering: <code>optimal_separates_far</code> , <code>sepFar_of_optimal</code>
Theorem 3	–	
Proposition 4(i)	full	<code>star_row_valid</code> (star rows in x -form, far-separating clusterings)
Proposition 4(ii)	partial	bad triangles and induced stars: <code>bad_triangle_costIn</code> , <code>star_bound</code> ; packings: <code>packing_bound</code>
rows (1)–(2)	full	<code>sep_triangle</code> , <code>sep_far_triangle</code>
Theorem 5(a)	full	<code>far_dual_bound</code> , <code>far_dual_le_opt</code> (bound on P for rows valid for far-separating clusterings, $\leq$ the cost of every clustering); <code>cost_eq_supp</code>
Theorem 5(b,c), Theorem 7, Corollary 8	–	
Theorem 9	(a) reduction, (b) full, (c) –	(a) as Theorem 13; (b) is Theorem 5(a)
Proposition 6	partial	the inequality evaluated by the checker, given valid rows: <code>far_dual_bound</code> , <code>far_dual_le_opt</code> ; the validity procedures (enumeration, star closed forms), the reader, the translation from ordered to unordered pairs and the rounding up are not formalised
Lemma 11	full	<code>twins_split_improvable</code> , <code>optimal_keeps_twins</code>
Pivot keeps twins together contraction (C1)	partial	one round: <code>pivot_round_twins</code>
move and swap formulas (C2)	partial	cost identity: <code>cost_contract</code>
Theorem 12	full	<code>cost_move</code> , <code>costW_move</code> , <code>costW_swap</code> , <code>cost_update</code>
		any pairwise-additive cost: <code>crossoverP_le</code> ; unweighted: <code>crossover_le</code> ; weighted: <code>crossoverW_le</code> ; relabelling: <code>cost_comp_injective</code> , <code>costW_comp_injective</code>
Theorem 13	reduction	<code>reach_le</code> , <code>output_le</code> , <code>search_guarantee</code> : non-worsening replacements keep a member no worse than the seed; the Pivot guarantee [3] is a hypothesis

Table B2 The ruin-and-recreate packing followed by block LPs against that packing alone with the same nominal budget of 1,200 s, one run each (seed 0), on the SNAP graphs with a support. m/n : average degree divided by two; t : measured running time; difference: of the packing alone relative to pack.+LP; largest checked: the packing alone gives the largest checked bound of our runs. Upper block: development graphs; lower block: held-out graphs.

graph	m/n	pack.+LP		packing, 1,200 s		difference (%)	largest checked
		LB	t (s)	LB	t (s)		
ca-GrQc	2.8	6,034	141	6,006	82	-0.46	
ca-HepTh	2.6	15,725	956	15,730	866	+0.03	yes
ca-HepPh	9.9	43,740	726	43,139	185	-1.37	
ca-AstroPh	10.6	119,101	762	118,233	181	-0.73	
ca-CondMat	4.0	55,910	719	55,636	127	-0.49	
email-Enron	5.0	140,862	2188	140,959	2043	+0.07	yes
loc-Brightkite	3.7	164,106	1516	164,211	1329	+0.06	yes
BitcoinOTC+	3.3	15,661	809	15,660	346	-0.01	
BitcoinAlpha+	3.5	10,820	711	10,805	317	-0.14	
com-Amazon	2.8	576,376	758	579,382	251	+0.52	yes
com-DBLP	3.3	589,275	823	591,866	286	+0.44	yes
email-Eu-core	16.0	11,651	902	11,455	636	-1.68	
facebook	21.8	44,025	798	43,076	267	-2.16	
wiki-Vote	14.2	85,548	1098	85,533	455	-0.02	
cit-HepPh	12.2	324,608	952	324,653	367	+0.01	yes
p2p-Gnutella31	2.4	130,326	689	130,546	540	+0.17	yes
amazon0302	3.4	508,994	709	515,478	252	+1.27	yes
roadNet-PA	1.4	905,670	662	890,136	148	-1.72	

them it is the largest checked bound of our runs (marked § in Table 6). Neither use of the time dominates, and the split between the two phases remains a tuning choice that we did not optimise; the certified gaps of Table 6 use the larger of the two.

Appendix C PXMmem and its comparison with KaPoCE

A certificate says how far a clustering may be from optimal; how close it actually is depends on the clustering. To test the certificates against strong solutions, and because CertiFlip spends much of its budget on the bound, we also use a second primal solver, *PXMmem*. It rests on two elementary exact facts: twins can be contracted, and two clusterings can be recombined block by block of their overlap without losing anything. Both have close relatives in the literature, which we point out; we claim no novelty for either, only for their combination into a solver. *PXMmem* produces no bound of its own; its solutions are certified with the bounds of Section 3.

C.1 Critical-clique contraction

Two vertices $u \neq v$ are *twins* if $uv \in E^+$ and $N(u) \setminus \{v\} = N(v) \setminus \{u\}$, i.e. they have the same closed neighbourhood. The classes of twins are the *critical cliques* of Guo [49]. The following lemma is due to Guo [49] (see also Protti et al. [68]); we include the short proof because the implementation relies on its exact form.

Lemma 11 (49) *Let u, v be twins and $\mathcal{C}$ a clustering that separates them. Moving v into the cluster of u and moving u into the cluster of v change the cost by δ_1 and δ_2 with $\delta_1 + \delta_2 = -2$. Hence every optimal clustering keeps every critical clique inside one cluster.*

Proof Let $A \ni u$ and $B \ni v$ be the two clusters. For a cluster X let $f(X)$ be the number of mistakes on the pairs zw with $w \in V \setminus \{u, v\}$ when a vertex $z \in \{u, v\}$ is placed in X (all other vertices staying where they are). It is the same for u and v because they have the same neighbours in $V \setminus \{u, v\}$. The moves do not change $A \setminus \{u, v\}$ and $B \setminus \{u, v\}$. Besides the cut positive pair uv , the current cost of the pairs involving u or v is $f(A) + f(B)$; after the first move it is $2f(A)$, after the second $2f(B)$. So $\delta_1 + \delta_2 = 2f(A) + 2f(B) - 2(f(A) + f(B) + 1) = -2$ and one of the moves is a strict improvement. $\square$

Contracting every critical clique K to a node of size $s_K = |K|$ gives a weighted instance on the node set U : two nodes $K \neq L$ carry the weight $w_{KL} = s_K s_L$ if K and L are adjacent and 0 otherwise, and the cost of a clustering c of U is

$$\text{cost}_w(c) = \sum_{\{K, L\}: c(K)=c(L)} (s_K s_L - w_{KL}) + \sum_{\{K, L\}: c(K) \neq c(L)} w_{KL}. \quad (\text{C1})$$

The pairs inside a critical clique are positive and never cut. Hence the cost of the expanded clustering on G^+ equals $\text{cost}_w(c)$, and by Lemma 11 an optimal clustering of the contracted instance expands to an optimal clustering of G^+ . Merging vertices into weighted nodes in this way is standard in cluster editing [49, 69]. On the collaboration graphs the contraction removes 13%–22% of the vertices. Pivot never separates twins either: whether a vertex joins the cluster of the pivot p depends only on its closed neighbourhood, so each round treats twins alike. A Pivot clustering therefore maps to the contracted instance without changing its cost.

C.2 Partition crossover

Figure C1 illustrates the theorem below on two Pivot clusterings of the neighbourhood of Figure 2.

Theorem 12 (partition crossover) *Let the cost of a clustering be any sum of pair terms that depend only on whether the two vertices of the pair share a cluster (this covers cost and the weighted cost (C1)). Let a and b be clusterings and let H be the bipartite graph whose nodes are the clusters of a and of b , with an edge between $A \in a$ and $B \in b$ whenever $A \cap B \neq \emptyset$. Each connected component of H covers a vertex set X , the block, which is a union of clusters of a and also a union of clusters of b . For a clustering c let $\text{cost}_X(c)$ count the mistakes on the pairs inside X . Choose, independently in every block, the clusters of b if $\text{cost}_X(b) < \text{cost}_X(a)$ and those of a otherwise. The resulting clustering χ satisfies*

$$\text{cost}(\chi) = \text{cost}(a) - \sum_X \max(0, \text{cost}_X(a) - \text{cost}_X(b)) \leq \min(\text{cost}(a), \text{cost}(b)).$$

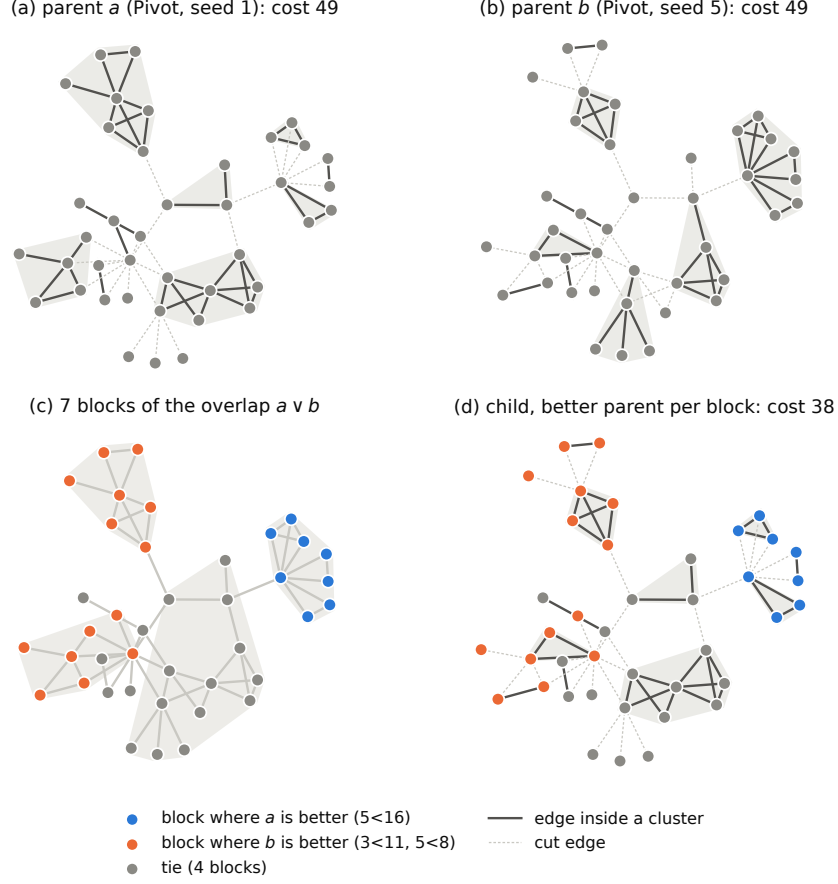


Fig. C1 Partition crossover (Theorem 12) computed by the released code. (a), (b): two Pivot clusterings of the neighbourhood of Figure 2, both of cost 49; shaded regions are clusters. (c): the connected components of the overlap graph H split the vertices into seven blocks, and on each block the two parents can be compared on their own. (d): taking the better parent on every block gives a clustering of cost 38, better than both parents.

Proof A pair u, v in different blocks is separated by a , by b and by χ , because each of their clusters lies inside one block. It therefore contributes the same amount to all three costs. A pair inside a block X is treated by χ exactly as by the parent chosen for X . Summing over the pairs gives the equality. Every term of the sum is non-negative, and exchanging the roles of a and b gives the bound by $\text{cost}(b)$. $\square$

The blocks are the connected components of a bipartite graph with at most n edges, and the block costs need one pass over E^+ . Hence χ is computed in $O(n + m)$ time. The blocks form the join $a \vee b$ of the two partitions in the partition lattice: the finest partition whose parts are unions of clusters of a and also of b . It is the finest partition for which every block-wise choice between the parents yields each parent as one of the choices and leaves the pairs between blocks separated in both. Decomposing

two parents into independent components and taking the better parent in each is the partition crossover of Whitley et al. [86, 87], Tinós et al. [88] for tours and of Tinós et al. [89] for pseudo-Boolean functions; recombination through the overlap of groups is common in grouping genetic algorithms [90, 91], memetic graph partitioning and clustering [92, 93] and ensemble clustering [94], and fusion moves [67] combine proposals optimally within a restricted space. Theorem 12 combines *any* two clusterings, including clusterings produced by different solvers. On our graphs, two independent runs typically differ in thousands of blocks, most of them ties, and each run wins some of the others.

C.3 Annealing moves

The search moves nodes of the contracted instance. Let $S_X = \sum_{K \in X} s_K$ be the size of a cluster X and $w(K, X) = \sum_{L \in X \setminus \{K\}} w_{KL}$. Moving a node K of size s from its cluster A to a cluster B changes cost_w by

$$\delta = s(S_B - S_A + s) + 2(w(K, A) - w(K, B)). \quad (\text{C2})$$

Exchanging $K \in A$ with $L \in B$ changes it by the sum of two such terms, where the second one uses $S_A - s_K$, $S_B + s_K$, $w(L, B) + w_{KL}$ and $w(L, A) - w_{KL}$. Both are evaluated in $O(\deg K + \deg L)$ time. The simulated annealing [95] proposes the move to the cluster of a random neighbour, the best move, or a move to a new cluster, and accepts with the Metropolis rule under a geometric cooling schedule. It records the best state with an undo journal, so a new best state costs $O(1)$ instead of a copy of the labelling. This alone made the annealing 7.3 times faster on **com-Amazon**.

Region-wise acceptance.

Theorem 12 turns annealing into an iterated local search with a decomposed acceptance test. From the current clustering x we anneal a copy, starting at a raised temperature, down to its final state y , and set $x \leftarrow \chi(x, y)$. Every block in which y improved on x is kept, even if y is worse overall, and the cost of x never increases.

Localized annealing with rollback.

A step grows a region of at most 30 nodes by breadth-first search around a centre, anneals only the nodes of the region from a high to a low temperature, finishes with greedy best moves, and undoes all moves if their total change is positive. The undo is exact: moves are reversed in LIFO order together with the free list of cluster labels. Half of the centres are drawn with probability proportional to the degree. Comparing our solutions with those of KaPoCE block by block showed why: the blocks in which KaPoCE was better all contained a vertex of high degree.

C.4 The memetic search

Algorithm 2 keeps a population of four clusterings. A crossover child enters the population only by replacing a member that is not cheaper, preferring the most similar members (crowding); the other two operators replace a member only if they improve

Algorithm 2 PXM_{Mem}(G^+, T)

```
1: contract the critical cliques
2: for  $i = 1, \dots, 4$  do
3:    $p_i \leftarrow \text{PIVOT}(G^+)$ ;  $f_i \leftarrow$  one round of  $\text{ITERATEDFLIP}(p_i)$ 
4:    $x_i \leftarrow$  the cheaper of  $p_i$  and  $f_i$  on the contracted nodes; anneal  $x_i$ 
5: end for
6: while time remains do
7:   choose an operator by an  $\varepsilon$ -greedy bandit on the recent gain per second
8:   crossover: parents  $a, b$  by binary tournaments;  $z \leftarrow \chi(a, b)$ ;
      contract the blocks common to  $a$  and  $b$  and run a multilevel local search from
       $z$ ;
      anneal on the nodes where  $a$  and  $b$  disagree; accept region-wise; crowding
      replacement
9:   PX-annealing step on a member (temperature chosen by a second bandit)
10:  localized annealing with rollback on a member
11: end while
12: return the cheapest member, expanded to  $V$ 
```

it. Contracting the blocks common to both parents and continuing with a multilevel search from the recombined clustering follows the recombination of memetic graph partitioners and clusterers [9, 92, 93]. The flipping round of line 3 costs a few full local-search passes whatever its time limit; with budgets below 300 s the seeds are therefore Pivot clusterings only, and no further seed is generated once half of the budget is used.

Theorem 13 *Let $\mathcal{C}$ be the output of Algorithm 2 and p_1 the first Pivot clustering. Then $\text{cost}(\mathcal{C}) \leq \text{cost}(p_1)$ for every outcome of the random choices, and hence $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 3 \text{OPT}$. Together with a lower bound LB from Section 3.2, $\text{cost}(\mathcal{C})/\lceil \text{LB} \rceil$ bounds the ratio achieved on the instance.*

The guarantee is inherited from the Pivot seed, as for any procedure that starts from a Pivot clustering and never makes its best solution worse; we state it because every operator of the search must respect it, and this is what the formal development checks.

Proof The seed x_1 costs at most $\text{cost}(p_1)$: Pivot does not separate twins, so its contraction preserves its cost, and x_1 is the cheaper of the two candidates. The annealing returns the cheapest state of a trajectory that starts at its input; region-wise acceptance does not increase the cost by Theorem 12; the localized annealing keeps a step only if its total change is non-positive; and every replacement is by a clustering that is not more expensive than the member it replaces. By induction over the replacements, some member always costs at most $\text{cost}(x_1)$, so the cheapest final member costs at most $\text{cost}(p_1)$. Taking expectations and applying the Pivot guarantee [3] gives the second claim. The last claim is Theorem 5(a). $\square$

Table C3 PXMmem against KaPoCE on the twelve held-out SNAP graphs, $T = 600$ s (primary budget), ten seeds per graph. W/T/L: runs in which PXMmem is better, equal, worse. median, HL: median and Hodges–Lehmann estimate of $(\text{cost}_{\text{PXMmem}} - \text{cost}_{\text{KaPoCE}}) / \text{cost}_{\text{KaPoCE}}$ in percent, negative when PXMmem is better, with the exact 95% confidence interval; p : exact sign test over the seeds; p_{Holm} : Holm-adjusted over the twelve graphs. Last row: the pre-registered tests over the twelve graph medians.

graph	runs	W/T/L	median (%)	HL (%)	[95% CI]	p	p_{Holm}
email-Eu-core	10	0/10/0	+0.000	+0.000	[+0.000, +0.000]	1.000	1.000
facebook	10	0/10/0	+0.000	+0.000	[+0.000, +0.000]	1.000	1.000
wiki-Vote	10	9/0/1	−0.016	−0.015	[−0.023, −0.007]	0.021	0.193
cit-HepTh	10	9/0/1	−0.016	−0.016	[−0.024, −0.008]	0.021	0.193
cit-HepPh	10	9/0/1	−0.019	−0.019	[−0.029, −0.010]	0.021	0.193
p2p-Gnutella31	10	0/7/3	+0.000	+0.000	[+0.000, +0.000]	0.250	1.000
soc-Slashdot0902	10	1/1/8	+0.003	+0.003	[+0.001, +0.005]	0.039	0.234
loc-Gowalla	10	10/0/0	−0.010	−0.011	[−0.016, −0.006]	0.002	0.023
email-EuAll	10	4/0/6	+0.003	+0.002	[−0.001, +0.005]	0.754	1.000
amazon0302	10	0/0/10	+0.001	+0.001	[+0.001, +0.002]	0.002	0.023
web-NotreDame	10	3/0/7	+0.001	+0.001	[−0.002, +0.003]	0.344	1.000
roadNet-PA	10	0/0/10	+0.084	+0.083	[+0.064, +0.099]	0.002	0.023
graph medians: 4 better, 3 equal, 5 worse; Wilcoxon $p = 0.734$, sign test $p = 1.000$							

C.5 Comparison with KaPoCE

Protocol.

PXMmem and KaPoCE are compared under a protocol that was fixed before the runs (`experiments/heldout.md`). On one machine, the two solvers run one after the other in an order drawn from a hash of (graph, seed), each on the same logical core while the machine is otherwise idle. Both read the same PACE-format text file inside their measured time. KaPoCE’s only changes are a seed and its two internal time limits, set to $\frac{590}{600}T$ and $\frac{90}{600}T$ (the ratio of its PACE configuration); it is stopped with SIGTERM at T and its output is validated. PXMmem is scored at KaPoCE’s elapsed wall-clock time, read from its improvement history, so the comparison is at equal elapsed time; its value at T is also recorded. Invalid KaPoCE output counts as a PXMmem win, as in PACE, and is reported separately. Budgets are $T = 600$ s (primary), 150 s and 60 s, with ten seeds on each held-out SNAP graph and one run on each PACE heuristic instance at 600 s.

Statistics.

The unit of inference is the graph. For each graph we take the median over seeds of the relative difference $(\text{cost}_{\text{PXMmem}} - \text{cost}_{\text{KaPoCE}}) / \text{cost}_{\text{KaPoCE}}$ and apply the two-sided Wilcoxon signed-rank test [96] and the sign test to the twelve medians. Per graph we report the exact sign test over seeds with Holm’s correction [97] over graphs, noting that with ten seeds only a 10:0 split can be significant, and the Hodges–Lehmann estimate [98] of the paired difference with its exact confidence interval. We do not claim equivalence from non-significance. Performance profiles [99] summarise the PACE heuristic track.

Table C4 As Table C3, $T = 150$ s.

graph	runs	W/T/L	median (%)	HL (%)	[95% CI]	p	p_{Holm}
email-Eu-core	10	0/8/2	+0.000	+0.000	[+0.000, +0.004]	0.500	1.000
facebook	10	1/8/1	+0.000	+0.000	[-0.003, +0.004]	1.000	1.000
wiki-Vote	10	9/0/1	-0.010	-0.012	[-0.019, +0.002]	0.021	0.150
cit-HepTh	10	9/0/1	-0.020	-0.018	[-0.031, +0.008]	0.021	0.150
cit-HepPh	10	7/0/3	-0.010	-0.009	[-0.021, +0.009]	0.344	1.000
p2p-Gnutella31	10	0/5/5	+0.000	+0.000	[+0.000, +0.002]	0.062	0.312
soc-Slashdot0902	10	0/0/10	+0.007	+0.007	[+0.005, +0.009]	0.002	0.023
loc-Gowalla	10	4/0/6	+0.003	+0.004	[-0.001, +0.007]	0.754	1.000
email-EuAll	10	10/0/0	-0.007	-0.008	[-0.017, -0.003]	0.002	0.023
amazon0302	10	0/0/10	+0.007	+0.007	[+0.005, +0.008]	0.002	0.023
web-NotreDame	10	0/0/10	+0.021	+0.022	[+0.017, +0.028]	0.002	0.023
roadNet-PA	10	0/0/10	+0.463	+0.439	[+0.365, +0.517]	0.002	0.023
graph medians: 4 better, 2 equal, 6 worse; Wilcoxon $p = 0.846$, sign test $p = 0.754$							

Table C5 As Table C3, $T = 60$ s.

graph	runs	W/T/L	median (%)	HL (%)	[95% CI]	p	p_{Holm}
email-Eu-core	10	0/5/5	+0.004	+0.004	[+0.000, +0.008]	0.062	0.375
facebook	10	4/4/2	+0.000	-0.009	[-0.280, +0.001]	0.688	1.000
wiki-Vote	10	4/0/6	+0.008	+0.010	[-0.003, +0.028]	0.754	1.000
cit-HepTh	10	6/1/3	-0.015	-0.014	[-0.029, +0.007]	0.508	1.000
cit-HepPh	10	3/0/7	+0.008	+0.009	[-0.006, +0.022]	0.344	1.000
p2p-Gnutella31	10	0/0/10	+0.002	+0.002	[+0.002, +0.003]	0.002	0.023
soc-Slashdot0902	10	0/0/10	+0.027	+0.026	[+0.020, +0.031]	0.002	0.023
loc-Gowalla	10	0/0/10	+0.028	+0.027	[+0.014, +0.038]	0.002	0.023
email-EuAll	10	2/0/8	+0.026	+0.023	[+0.008, +0.038]	0.109	0.547
amazon0302	10	0/0/10	+0.029	+0.030	[+0.022, +0.039]	0.002	0.023
web-NotreDame	10	0/0/10	+0.072	+0.068	[+0.062, +0.075]	0.002	0.023
roadNet-PA	10	0/0/10	+2.299	+2.146	[+1.685, +2.595]	0.002	0.023
graph medians: 1 better, 1 equal, 10 worse; Wilcoxon $p = 0.010$, sign test $p = 0.012$							

Primary budget.

At 600 s (Table C3, 120 paired runs) the graph medians favour PXMEm on 4 graphs and KaPoCE on 5, with 3 ties. Neither pre-registered test rejects equality of the two solvers (Wilcoxon $p = 0.734$, sign test $p = 1.000$). After Holm's correction PXMEm is better on **loc-Gowalla** and KaPoCE on **amazon0302** and **roadNet-PA**; PXMEm wins nine of ten seeds on **wiki-Vote**, **cit-HepTh** and **cit-HepPh**, which is not significant after the correction. All differences are small: the largest graph median is 0.08%, and on the held-out graphs with a certificate the certified gap of the best solution is at least 0.63% (Table 6). The certificates therefore cannot separate the two solvers, and neither solver is shown to be near-optimal on these graphs beyond what Table 6 certifies. KaPoCE returned a valid solution in every run (0 forfeits). Scoring PXMEm at T instead of at KaPoCE's elapsed time gives 4 better and 5 worse graph medians (Wilcoxon $p = 0.734$). Many per-seed differences are zero or tied on some graphs (**email-Eu-core**, **facebook**, **p2p-Gnutella31**); there the Hodges-Lehmann interval degenerates and its coverage is not exact, and zero graph medians are dropped from

Table C6 PACE 2021 heuristic track, $T = 600$ s, one run per instance: PXMem against KaPoCE by instance size. W/T/L: instances on which PXMem is better, equal, worse; Σ : total difference of the costs; median: of the relative differences in percent.

vertices	instances	W/T/L	Σ PXMem–KaPoCE	median (%)
$< 10^3$	93	0/91/2	+5	+0.0000
10^3 – 10^4	78	7/65/6	+93	+0.0000
10^4 – 10^5	8	6/2/0	–257	–0.0028
$\geq 10^5$	11	6/0/5	+5344	–0.0017
without dev. twins	190	19/158/13	+5185	+0.0000
development twins	10	5/2/3	+219	–0.0032

the Wilcoxon test, which the analysis plan did not specify. We do not claim that the two solvers are equivalent: the study was not designed as an equivalence test.

Shorter budgets.

KaPoCE did not stop by itself and was terminated by SIGTERM in 0, 18 and 46 of the 120 runs at 600 s, 150 s and 60 s; it then writes its best solution, which can take some seconds, while PXMem is scored at KaPoCE’s elapsed time. This favours KaPoCE. At 150 s (Table C4) the picture is similar (4 better, 2 equal, 6 worse; Wilcoxon $p = 0.846$), with larger losses of PXMem on the largest graphs, up to 0.46% on **roadNet-PA**. At 60 s (Table C5) KaPoCE is better on 10 of the 12 graph medians (Wilcoxon $p = 0.010$). PXMem is a population method, and a short budget leaves few recombinations on graphs with 10^5 vertices or more; we did not analyse this further. PXMem is thus not a replacement for KaPoCE at short budgets, and at the primary budget the two are not distinguishable by our tests.

PACE heuristic track.

On the 200 instances of the PACE 2021 heuristic track, one run per instance at 600 s under the same protocol, the instance is the unit. The 10 instances that coincide with development graphs are reported separately (Table C6). On the other 190 instances PXMem is better on 19, equal on 158 and worse on 13 (Wilcoxon $p = 0.789$, sign test $p = 0.377$); KaPoCE returned no valid solution on 0 instances, which count as PXMem wins as in PACE. The counts hide the sizes of the differences: summed over the 190 instances the costs favour KaPoCE by 5185, and the eleven instances with 10^5 vertices or more contribute 5344 of this (PXMem better on six of them, worse on five). PXMem records its first solution only after its initialisation; on 1 small instance KaPoCE stopped before that, and it counts as a KaPoCE win under the protocol. Figure C2 shows the performance profile.

Appendix D Development log

This appendix records which data informed which decision, so that the reader can judge which results are out of sample.

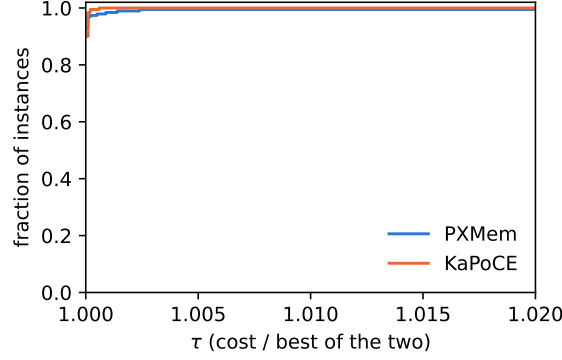


Fig. C2 Performance profile [99] on the PACE heuristic track without the development twins: fraction of instances on which a solver’s cost is within a factor τ of the better of the two.

- *Development data.* The fifteen SNAP graphs of Table 1 marked “dev” and the 200 PACE 2021 exact-track instances. All parameters of the bound (block size 2500, $|H| \leq 8$ for local subgraph rows, ruin regions of about 12 vertices, a target of half of the remaining budget for the bound), of the LNS ($|B| \leq 60$) and of PXMmem (population 4, regions of at most 30 nodes, bandit settings, the degree bias of the region centres) were chosen by looking at runs on these instances. Several PXMmem components were added after inspecting where KaPoCE’s solutions were better than ours on development graphs (Section C).
- *Earlier comparisons.* A head-to-head comparison with KaPoCE on the development graphs and on the PACE heuristic track was carried out before the held-out set was fixed. Its raw results remain in the repository; it is not the basis of any claim in this paper, because the same graphs were used for tuning and because it measured the two solvers asymmetrically (KaPoCE’s time included reading the input, PXMmem’s did not, and the order was fixed).
- *Machines.* Earlier runs of the PACE exact track on a 4-core workstation with four runs in parallel, also with checked certificates, proved optimality on 111 of the 173 solved instances. All numbers in this paper are from the runs on the Colab machines described in Section 6.1, which give 106; we report the latter because the other bounds of Table 2 were computed on the same machines and because their certificates are the archived ones.
- *Held-out data.* The twelve SNAP graphs marked “held-out” in Table 1 and the analysis plan of Section 6.1 were fixed and committed to the repository (`experiments/heldout.md`) before any bound was computed on them and before any run of the protocol. The plan states that no earlier run of either solver on them existed; this is not correct. Seven of the held-out graphs are PACE heuristic-track instances, and all 200 heuristic-track instances had been run once with both solvers in the earlier comparison above. That comparison was made after the last change to the solver code (the code has not changed since), and it informed no parameter, but its results were known when the held-out set was chosen. The ten heuristic-track

instances that coincide with development graphs are reported separately; the other 190 were not used for tuning, but they are not held out in the strict sense.

- *Gap-guided search.* We did not evaluate the gap-guided choice of blocks and LNS neighbourhoods separately in the final experiments. In the development runs (`results/ablation_lb.csv`, five development graphs, 4-core workstation, 300 s), gap-guided blocks after the METIS sweeps changed the bound by -0.004% to $+0.03\%$; the same runs gave larger bounds with block LPs after a packing than with a packing alone on all five graphs.
- *Headline bound.* The decision to report the largest of the CertiFlip bound, the separate star-packing bound and the long packing followed by block LPs on the SNAP graphs was taken after the packing runs were complete, and the last procedure and the packing runs on the graphs above the support threshold were added after the second round of review; all three are valid, and all are reported per graph. The result files of the long-packing runs record the commit of the code package they were started from, which predates the script mode that ran them; the solver code was not changed between the two.
- *Star local search.* The star local search was written after all other results in this paper were known, when a rerun of the lower-bound program of the branch-and-bound had shown that its root packing was far stronger than our ruin-and-recreate packing on the same rows. It follows the heuristic of that program; the program’s code was read to understand the heuristic, and none of it was copied (the solver is GPL-licensed, ours is BSD). While testing it on nine PACE exact instances we found and fixed one error (trying and undoing a leaf extension invalidated the remaining candidates of that star), and before the final runs it was tried for 60 to 120 s on nine SNAP graphs, seven of them held out (`facebook`, `email-Eu-core`, `wiki-Vote`, `cit-HepPh`, `amazon0302`, `roadNet-PA`, `loc-Gowalla`) and two development graphs (`ca-HepPh`, `com-DBLP`); nothing was changed after these trials. Its parameters (at most five unimproved rounds per improving one, probability 0.8 for the plateau move, 16 candidates per freed pair) are those of the branch-and-bound or were not tuned; the time limits, 60 s on PACE and 600 s on SNAP, are those of the CertiFlip runs. All reported runs are from one version of its code, except the leaf cap on two graphs, which was added after their first runs had been killed. Its runs are therefore not out of sample in the sense of the held-out protocol, which was fixed for the head-to-head comparison.
- *Controls after the third round of review.* A review of the version built around the star local search pointed out that its advantage on the PACE exact track had not been separated from its longer running time; a reviewer reran it with the stopping rule of the branch-and-bound and found that the advantage disappears. We then added the runs with that rule, the further seeds, the 300 s, 1,200 s and 3,600 s runs and the logging of the trajectory, all with the code of the earlier runs (the trajectory was already written to the error stream and is now kept; the stopping rule was already an option), and rewrote the claims accordingly. The same review found the unguarded 64-bit sum in the row check of the checker (Section 4); it was fixed before these runs, and all certificates were re-checked with the fixed version. PXMem and the head-to-head comparison were moved to Appendix C without changes to their results.

References

- [1] Bansal, N., Blum, A., Chawla, S.: Correlation clustering. *Machine Learning* **56**(1-3), 89–113 (2004) <https://doi.org/10.1023/B:MACH.0000033116.57574.95> . Conference version: FOCS 2002, pp. 238–247, doi:10.1109/SFCS.2002.1181947
- [2] Charikar, M., Guruswami, V., Wirth, A.: Clustering with qualitative information. *Journal of Computer and System Sciences* **71**(3), 360–383 (2005) <https://doi.org/10.1016/j.jcss.2004.10.012> . Conference version: FOCS 2003, pp. 524–533, doi:10.1109/SFCS.2003.1238225
- [3] Ailon, N., Charikar, M., Newman, A.: Aggregating inconsistent information: Ranking and clustering. *Journal of the ACM* **55**(5), 1–27 (2008) <https://doi.org/10.1145/1411509.1411513> . Article 23. Conference version: STOC 2005, pp. 684–693, doi:10.1145/1060590.1060692
- [4] Chawla, S., Makarychev, K., Schramm, T., Yaroslavtsev, G.: Near optimal LP rounding algorithm for correlation clustering on complete and complete k-partite graphs. In: *Proceedings of the 47th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 219–228 (2015). <https://doi.org/10.1145/2746539.2746604>
- [5] Cao, N., Cohen-Addad, V., Lee, E., Li, S., Newman, A., Vogl, L.: Understanding the cluster linear program for correlation clustering. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 1605–1616 (2024). <https://doi.org/10.1145/3618260.3649749> . Conference version claims 1.437; arXiv v3 (2025) reports a gap in that proof and proves 1.485
- [6] García-Soriano, D., Schohn, A.: Approximate Dual Separation for the Cluster LP: a 1.387 approximation for Correlation Clustering. arXiv preprint arXiv:2607.27829 (2026)
- [7] Bläsius, T., Fischbeck, P., Gottesbüren, L., Hamann, M., Heuer, T., Spinner, J., Weyand, C., Wilhelm, M.: PACE solver description: KaPoCE: A heuristic cluster editing algorithm. In: *16th International Symposium on Parameterized and Exact Computation (IPEC 2021)*. LIPIcs, vol. 214, pp. 1–4 (2021). <https://doi.org/10.4230/LIPIcs.IPEC.2021.31> . Article 31
- [8] Kellerhals, L., Koana, T., Nichterlein, A., Zschoche, P.: The PACE 2021 parameterized algorithms and computational experiments challenge: Cluster editing. In: Golovach, P.A., Zehavi, M. (eds.) *16th International Symposium on Parameterized and Exact Computation (IPEC 2021)*. LIPIcs, vol. 214, pp. 1–18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). <https://doi.org/10.4230/LIPIcs.IPEC.2021.26> . Article 26
- [9] Hausberger, F., Faraj, M.F., Schulz, C.: Scalable multilevel and memetic signed

- graph clustering. In: Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX), pp. 81–94 (2025). <https://doi.org/10.1137/1.9781611978339.7>
- [10] Bläsius, T., Fischbeck, P., Gottesbüren, L., Hamann, M., Heuer, T., Spinner, J., Weyand, C., Wilhelm, M.: A branch-and-bound algorithm for cluster editing. In: 20th International Symposium on Experimental Algorithms (SEA 2022). LIPIcs, vol. 233, pp. 1–19 (2022). <https://doi.org/10.4230/LIPIcs.SEA.2022.13> . Article 13
 - [11] Veldt, N., Gleich, D.F., Wirth, A., Saunderson, J.: Metric-constrained optimization for graph clustering algorithms. *SIAM Journal on Mathematics of Data Science* **1**(2), 333–355 (2019) <https://doi.org/10.1137/18M1217152> [arXiv:1806.01678](https://arxiv.org/abs/1806.01678)
 - [12] Ruggles, C., Veldt, N., Gleich, D.F.: A parallel projection method for metric constrained optimization. In: Proceedings of the SIAM Workshop on Combinatorial Scientific Computing (CSC), pp. 43–53 (2020). <https://doi.org/10.1137/1.9781611976229.5>
 - [13] Veldt, N.: Correlation clustering via strong triadic closure labeling: Fast approximation algorithms and practical lower bounds. In: Proceedings of the 39th International Conference on Machine Learning (ICML). PMLR, vol. 162, pp. 22060–22083 (2022)
 - [14] Gottesbüren, L., Hamann, M., Schoch, P., Strasser, B., Wagner, D., Zühlsdorf, S.: Engineering exact quasi-threshold editing. In: 18th International Symposium on Experimental Algorithms (SEA 2020). LIPIcs, vol. 160, pp. 10–11014. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, ??? (2020). <https://doi.org/10.4230/LIPIcs.SEA.2020.10>
 - [15] McConnell, R.M., Mehlhorn, K., Näher, S., Schweitzer, P.: Certifying algorithms. *Computer Science Review* **5**(2), 119–161 (2011) <https://doi.org/10.1016/j.cosrev.2010.09.009>
 - [16] Neumaier, A., Shcherbina, O.: Safe bounds in linear and mixed-integer linear programming. *Mathematical Programming* **99**(2), 283–296 (2004) <https://doi.org/10.1007/s10107-003-0433-3>
 - [17] Steffy, D.E., Wolter, K.: Valid linear programming bounds for exact mixed-integer programming. *INFORMS Journal on Computing* **25**(2), 271–284 (2013) <https://doi.org/10.1287/ijoc.1120.0501>
 - [18] Cheung, K.K.H., Gleixner, A., Steffy, D.E.: Verifying integer programming results. In: Integer Programming and Combinatorial Optimization (IPCO 2017). Lecture Notes in Computer Science, vol. 10328, pp. 148–160. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59250-3_13

- [19] Wetzler, N., Heule, M.J.H., Hunt, W.A.: DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In: Theory and Applications of Satisfiability Testing – SAT 2014. Lecture Notes in Computer Science, vol. 8561, pp. 422–429. Springer, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-319-09284-3_31
- [20] Gocht, S., Nordström, J.: Certifying parity reasoning efficiently using pseudo-Boolean proofs. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 3768–3777. AAAI Press, Palo Alto, CA (2021). <https://doi.org/10.1609/aaai.v35i5.16494>
- [21] Cohen-Addad, V., Lee, E., Newman, A.: Correlation clustering with Sherali-Adams. In: 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS), pp. 651–661 (2022). <https://doi.org/10.1109/FOCS54457.2022.00068>
- [22] Cohen-Addad, V., Lee, E., Li, S., Newman, A.: Handling correlated rounding error via preclustering: A 1.73-approximation for correlation clustering. In: 64th IEEE Annual Symposium on Foundations of Computer Science (FOCS), pp. 1082–1104 (2023). <https://doi.org/10.1109/FOCS57990.2023.00065>
- [23] Cao, N., Cohen-Addad, V., Li, S., Lee, E., Lolck, D.R., Newman, A., Thorup, M., Vogl, L., Yan, S., Zhang, H.: Solving the correlation cluster LP in sublinear time. In: Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC), pp. 1154–1165 (2025). <https://doi.org/10.1145/3717823.3718181>
- [24] Cohen-Addad, V., Lolck, D.R., Pilipczuk, M., Thorup, M., Yan, S., Zhang, H.: Combinatorial correlation clustering. In: Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC), pp. 1617–1628 (2024). <https://doi.org/10.1145/3618260.3649712>
- [25] Cohen-Addad, V., Lattanzi, S., Mitrović, S., Norouzi-Fard, A., Parotsidis, N., Tarnawski, J.: Correlation clustering in constant many parallel rounds. In: Proceedings of the 38th International Conference on Machine Learning (ICML). PMLR, vol. 139, pp. 2069–2078 (2021)
- [26] Behnezhad, S., Charikar, M., Ma, W., Tan, L.-Y.: Almost 3-approximate correlation clustering in constant rounds. In: IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS), pp. 720–731 (2022). <https://doi.org/10.1109/FOCS54457.2022.00074>
- [27] Behnezhad, S., Charikar, M., Ma, W., Tan, L.-Y.: Single-pass streaming algorithms for correlation clustering. In: Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 819–849 (2023). <https://doi.org/10.1137/1.9781611977554.ch33>
- [28] Cambus, M., Kuhn, F., Lindy, E., Pai, S., Uitto, J.: A $(3 + \varepsilon)$ -approximate correlation clustering algorithm in dynamic streams. In: Proceedings of the 2024

- Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pp. 2861–2880 (2024). <https://doi.org/10.1137/1.9781611977912.101>
- [29] Cohen-Addad, V., Lattanzi, S., Maggiori, A., Parotsidis, N.: Dynamic correlation clustering in sublinear update time. In: Proceedings of the 41st International Conference on Machine Learning (ICML). PMLR, vol. 235, pp. 9230–9274 (2024)
 - [30] Demaine, E.D., Emanuel, D., Fiat, A., Immorlica, N.: Correlation clustering in general weighted graphs. *Theoretical Computer Science* **361**(2-3), 172–187 (2006) <https://doi.org/10.1016/j.tcs.2006.05.008>
 - [31] Böcker, S., Baumbach, J.: Cluster editing. In: The Nature of Computation. Logic, Algorithms, Applications (CiE 2013). Lecture Notes in Computer Science, vol. 7921, pp. 33–44. Springer, ??? (2013). https://doi.org/10.1007/978-3-642-39053-1_5
 - [32] Garg, N., Könemann, J.: Faster and simpler algorithms for multicommodity flow and other fractional packing problems. *SIAM Journal on Computing* **37**(2), 630–652 (2007) <https://doi.org/10.1137/S0097539704446232>
 - [33] Fleischer, L.K.: Approximating fractional multicommodity flow independent of the number of commodities. *SIAM Journal on Discrete Mathematics* **13**(4), 505–520 (2000) <https://doi.org/10.1137/S0895480199355754>
 - [34] Sonthalia, R., Gilbert, A.C.: Project and forget: Solving large-scale metric constrained problems. *Journal of Machine Learning Research* **23**(326), 1–54 (2022)
 - [35] Böcker, S., Briesemeister, S., Klau, G.W.: Exact algorithms for cluster editing: Evaluation and experiments. *Algorithmica* **60**(2), 316–334 (2011) <https://doi.org/10.1007/s00453-009-9339-7>
 - [36] Bläsius, T., Fischbeck, P., Gottesbüren, L., Hamann, M., Heuer, T., Spinner, J., Weyand, C., Wilhelm, M.: PACE solver description: The KaPoCE exact cluster editing algorithm. In: 16th International Symposium on Parameterized and Exact Computation (IPEC 2021). LIPIcs, vol. 214, pp. 1–3 (2021). <https://doi.org/10.4230/LIPIcs.IPEC.2021.27> . Article 27
 - [37] Bastos, L., Ochi, L.S., Protti, F., Subramanian, A., Martins, I.C., Pinheiro, R.G.S.: Efficient algorithms for cluster editing. *Journal of Combinatorial Optimization* **31**(1), 347–371 (2016) <https://doi.org/10.1007/s10878-014-9756-7>
 - [38] Bevern, R., Froese, V., Komusiewicz, C.: Parameterizing edge modification problems above lower bounds. *Theory of Computing Systems* **62**(3), 739–770 (2018) <https://doi.org/10.1007/s00224-016-9746-5>

- [39] Li, S., Pilipczuk, M., Sorge, M.: Cluster editing parameterized above modification-disjoint P_3 -packings. In: 38th International Symposium on Theoretical Aspects of Computer Science (STACS 2021). LIPIcs, vol. 187, pp. 49–14916. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, ??? (2021). <https://doi.org/10.4230/LIPIcs.STACS.2021.49>
- [40] Grötschel, M., Wakabayashi, Y.: A cutting plane algorithm for a clustering problem. *Mathematical Programming* **45**(1-3), 59–96 (1989) <https://doi.org/10.1007/BF01589097>
- [41] Grötschel, M., Wakabayashi, Y.: Facets of the clique partitioning polytope. *Mathematical Programming* **47**(1-3), 367–387 (1990) <https://doi.org/10.1007/BF01580870>
- [42] Miyauchi, A., Sukegawa, N.: Redundant constraints in the standard formulation for the clique partitioning problem. *Optimization Letters* **9**(1), 199–207 (2015) <https://doi.org/10.1007/s11590-014-0754-6>
- [43] Miyauchi, A., Sonobe, T., Sukegawa, N.: Exact clustering via integer programming and maximum satisfiability. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32 (2018). <https://doi.org/10.1609/aaai.v32i1.11519>
- [44] Koshimura, M., Watanabe, E., Sakurai, Y., Yokoo, M.: Concise integer linear programming formulation for clique partitioning problems. *Constraints* **27**(1-2), 99–115 (2022) <https://doi.org/10.1007/s10601-022-09326-z>
- [45] Irmay, J., Naumann, L.F., Andres, B.: Chorded cycle facets of the clique partitioning polytope. *arXiv preprint arXiv:2411.03407* (2024)
- [46] Sukegawa, N., Yamamoto, Y., Zhang, L.: Lagrangian relaxation and pegging test for the clique partitioning problem. *Advances in Data Analysis and Classification* **7**(4), 363–391 (2013) <https://doi.org/10.1007/s11634-013-0135-5>
- [47] Belyi, A., Sobolevsky, S., Kurbatski, A., Ratti, C.: Subnetwork constraints for tighter upper bounds and exact solution of the clique partitioning problem. *Mathematical Methods of Operations Research* **98**(2), 269–297 (2023) <https://doi.org/10.1007/s00186-023-00835-y> [arXiv:2110.05627](https://arxiv.org/abs/2110.05627)
- [48] Berg, J., Jarvisalo, M.: Cost-optimal constrained correlation clustering via weighted partial Maximum Satisfiability. *Artificial Intelligence* **244**, 110–142 (2017) <https://doi.org/10.1016/j.artint.2015.07.001>
- [49] Guo, J.: A more effective linear kernelization for cluster editing. *Theoretical Computer Science* **410**(8-10), 718–726 (2009) <https://doi.org/10.1016/j.tcs.2008.10.021>
- [50] Cao, Y., Chen, J.: Cluster editing: Kernelization based on edge cuts.

- Algorithmica **64**(1), 152–169 (2012) <https://doi.org/10.1007/s00453-011-9595-1> [arXiv:1008.4250](https://arxiv.org/abs/1008.4250)
- [51] Swoboda, P., Andres, B.: A message passing algorithm for the minimum cost multicut problem. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017), pp. 4990–4999 (2017). <https://doi.org/10.1109/CVPR.2017.530>
 - [52] Abbas, A., Swoboda, P.: RAMA: A rapid multicut algorithm on GPU. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022), pp. 8183–8192 (2022). <https://doi.org/10.1109/CVPR52688.2022.00802>
 - [53] Abbas, A., Swoboda, P.: ClusterFuG: Clustering fully connected graphs by multicut. In: Proceedings of the 40th International Conference on Machine Learning (ICML). PMLR, vol. 202, pp. 19–30 (2023)
 - [54] Lange, J.-H., Karrenbauer, A., Andres, B.: Partial optimality and fast lower bounds for weighted correlation clustering. In: Proceedings of the 35th International Conference on Machine Learning (ICML). PMLR, vol. 80, pp. 2892–2901 (2018)
 - [55] Lange, J.-H., Andres, B., Swoboda, P.: Combinatorial persistency criteria for multicut and max-cut. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6086–6095. IEEE, Piscataway, NJ (2019). <https://doi.org/10.1109/CVPR.2019.00625>
 - [56] Tourani, S., Shekhovtsov, A., Rother, C., Savchynskyy, B.: MPLP++: Fast, parallel dual block-coordinate ascent for dense graphical models. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, vol. 11208, pp. 264–281. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01225-0_16
 - [57] Lucena, A.: Non delayed relax-and-cut algorithms. Annals of Operations Research **140**(1), 375–410 (2005) <https://doi.org/10.1007/s10479-005-3977-1>
 - [58] Applegate, D., Díaz, M., Hinder, O., Lu, H., Lubin, M., O’Donoghue, B., Schudy, W.: Practical large-scale linear programming using primal-dual hybrid gradient. In: Advances in Neural Information Processing Systems 34 (NeurIPS 2021) (2021). [arXiv:2106.04756](https://arxiv.org/abs/2106.04756)
 - [59] Elsner, M., Schudy, W.: Bounding and comparing methods for correlation clustering beyond ILP. In: Proceedings of the Workshop on Integer Linear Programming for Natural Language Processing, pp. 19–27. Association for Computational Linguistics, Stroudsburg, PA (2009). <https://doi.org/10.3115/1611638.1611641>
 - [60] Veldt, N., Gleich, D.F., Wirth, A.: A correlation clustering framework for community detection. In: Proceedings of the 2018 World Wide Web Conference (WWW), pp. 439–448 (2018). <https://doi.org/10.1145/3178876.3186110>

- [61] Blondel, V.D., Guillaume, J.-L., Lambiotte, R., Lefebvre, E.: Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* **2008**(10), 10008 (2008) <https://doi.org/10.1088/1742-5468/2008/10/P10008>
- [62] Traag, V.A., Waltman, L., Eck, N.J.: From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* **9**, 5233 (2019) <https://doi.org/10.1038/s41598-019-41695-z>
- [63] Brusco, M.J., Köhn, H.-F.: Clustering qualitative data based on binary equivalence relations: Neighborhood search heuristics for the clique partitioning problem. *Psychometrika* **74**(4), 685–703 (2009) <https://doi.org/10.1007/s11336-009-9126-z>
- [64] Zhou, Y., Hao, J.-K., Goëffon, A.: A three-phased local search approach for the clique partitioning problem. *Journal of Combinatorial Optimization* **32**(2), 469–491 (2016) <https://doi.org/10.1007/s10878-015-9964-9>
- [65] Lu, Z., Zhou, Y., Hao, J.-K.: A hybrid evolutionary algorithm for the clique partitioning problem. *IEEE Transactions on Cybernetics* **52**(9), 9391–9403 (2022) <https://doi.org/10.1109/TCYB.2021.3051243>
- [66] Levinkov, E., Kirillov, A., Andres, B.: A comparative study of local search algorithms for correlation clustering. In: *German Conference on Pattern Recognition (GCPR)*. LNCS, vol. 10496, pp. 103–114 (2017). https://doi.org/10.1007/978-3-319-66709-6_9
- [67] Beier, T., Hamprecht, F.A., Kappes, J.H.: Fusion moves for correlation clustering. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3507–3516. IEEE, Piscataway, NJ (2015). <https://doi.org/10.1109/CVPR.2015.7298973>
- [68] Protti, F., Dantas da Silva, M., Szwarcfiter, J.L.: Applying modular decomposition to parameterized cluster editing problems. *Theory of Computing Systems* **44**(1), 91–104 (2009) <https://doi.org/10.1007/s00224-007-9032-7>
- [69] Böcker, S., Briesemeister, S., Bui, Q.B.A., Truss, A.: Going weighted: Parameterized algorithms for cluster editing. *Theoretical Computer Science* **410**(52), 5467–5480 (2009) <https://doi.org/10.1016/j.tcs.2009.05.006> . Conference version: COCOA 2008, pp. 1–12, doi:10.1007/978-3-540-85097-7_1
- [70] Böcker, S.: A golden ratio parameterized algorithm for cluster editing. *Journal of Discrete Algorithms* **16**, 79–89 (2012) <https://doi.org/10.1016/j.jda.2012.04.005> . Conference version: IWOCOA 2011, doi:10.1007/978-3-642-25011-8_7
- [71] Chen, J., Meng, J.: A 2k kernel for the cluster editing problem. *Journal of Computer and System Sciences* **78**(1), 211–220 (2012) <https://doi.org/10.1016/j.jcss.>

- [72] Wittkop, T., Baumbach, J., Lobo, F.P., Rahmann, S.: Large scale clustering of protein sequences with FORCE – A layout based heuristic for weighted cluster editing. *BMC Bioinformatics* **8**, 396 (2007) <https://doi.org/10.1186/1471-2105-8-396>
- [73] Wittkop, T., Emig, D., Lange, S., Rahmann, S., Albrecht, M., Morris, J.H., Böcker, S., Stoye, J., Baumbach, J.: Partitioning biological data with transitivity clustering. *Nature Methods* **7**(6), 419–420 (2010) <https://doi.org/10.1038/nmeth0610-419>
- [74] Applegate, D., Bixby, R., Chvátal, V., Cook, W.: TSP cuts which do not conform to the template paradigm. In: Jünger, M., Naddef, D. (eds.) *Computational Combinatorial Optimization: Optimal or Provably Near-Optimal Solutions*. LNCS, vol. 2241, pp. 261–303. Springer, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-45586-8_7
- [75] Karypis, G., Kumar, V.: A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing* **20**(1), 359–392 (1998) <https://doi.org/10.1137/S1064827595287997>
- [76] Huangfu, Q., Hall, J.A.J.: Parallelizing the dual revised simplex method. *Mathematical Programming Computation* **10**(1), 119–142 (2018) <https://doi.org/10.1007/s12532-017-0130-5>
- [77] Gocht, S., McBride, R., McCreesh, C., Nordström, J., Prosser, P., Trimble, J.: Certifying solvers for clique and maximum common (connected) subgraph problems. In: *Principles and Practice of Constraint Programming (CP 2020)*. Lecture Notes in Computer Science, vol. 12333, pp. 338–357. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-58475-7_20
- [78] Shaw, P.: Using constraint programming and local search methods to solve vehicle routing problems. In: *Principles and Practice of Constraint Programming – CP98*. LNCS, vol. 1520, pp. 417–431 (1998). https://doi.org/10.1007/3-540-49481-2_30
- [79] Danna, E., Rothberg, E., Le Pape, C.: Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming* **102**(1), 71–90 (2005) <https://doi.org/10.1007/s10107-004-0518-7>
- [80] Fischetti, M., Lodi, A.: Local branching. *Mathematical Programming* **98**(1-3), 23–47 (2003) <https://doi.org/10.1007/s10107-003-0395-5>
- [81] Leskovec, J., Krevl, A.: SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data> (2014)

- [82] Leskovec, J., Huttenlocher, D., Kleinberg, J.: Signed networks in social media. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI), pp. 1361–1370 (2010). <https://doi.org/10.1145/1753326.1753532>
- [83] Kumar, S., Spezzano, F., Subrahmanian, V.S., Faloutsos, C.: Edge weight prediction in weighted signed networks. In: IEEE 16th International Conference on Data Mining (ICDM), pp. 221–230 (2016). <https://doi.org/10.1109/ICDM.2016.0033>
- [84] Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Automated Deduction – CADE 28. Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_37
- [85] The mathlib Community: The Lean mathematical library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP), pp. 367–381 (2020). <https://doi.org/10.1145/3372885.3373824>
- [86] Whitley, D., Hains, D., Howe, A.: Tunneling between optima: Partition crossover for the traveling salesman problem. In: Proceedings of the Genetic and Evolutionary Computation Conference (GECCO), pp. 915–922 (2009). <https://doi.org/10.1145/1569901.1570026>
- [87] Whitley, D., Hains, D., Howe, A.: A hybrid genetic algorithm for the traveling salesman problem using generalized partition crossover. In: Parallel Problem Solving from Nature – PPSN XI. LNCS, vol. 6238, pp. 566–575. Springer, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15844-5_57
- [88] Tinós, R., Whitley, D., Ochoa, G.: A new generalized partition crossover for the traveling salesman problem: Tunneling between local optima. *Evolutionary Computation* **28**(2), 255–288 (2020) https://doi.org/10.1162/evco_a_00254
- [89] Tinós, R., Whitley, D., Chicano, F.: Partition crossover for pseudo-boolean optimization. In: Proceedings of the 2015 ACM Conference on Foundations of Genetic Algorithms XIII (FOGA), pp. 137–149. ACM, New York, NY (2015). <https://doi.org/10.1145/2725494.2725497>
- [90] Falkenauer, E.: Genetic Algorithms and Grouping Problems. John Wiley & Sons, Chichester, UK (1998)
- [91] Galinier, P., Hao, J.-K.: Hybrid evolutionary algorithms for graph coloring. *Journal of Combinatorial Optimization* **3**(4), 379–397 (1999) <https://doi.org/10.1023/A:1009823419804>
- [92] Sanders, P., Schulz, C.: Distributed evolutionary graph partitioning. In: Proceedings of the 14th Workshop on Algorithm Engineering and Experiments (ALENEX), pp. 16–29. SIAM, Philadelphia, PA (2012). <https://doi.org/10.1137/1.9781611972924.2>

- [93] Biedermann, S., Henzinger, M., Schulz, C., Schuster, B.: Memetic graph clustering. In: 17th International Symposium on Experimental Algorithms (SEA 2018). LIPIcs, vol. 103, pp. 1–15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2018). <https://doi.org/10.4230/LIPIcs.SEA.2018.3> . Article 3
- [94] Ovelgönne, M., Geyer-Schulz, A.: An ensemble learning strategy for graph clustering. In: Bader, D.A., Meyerhenke, H., Sanders, P., Wagner, D. (eds.) Graph Partitioning and Graph Clustering. Contemporary Mathematics, vol. 588, pp. 187–205. American Mathematical Society, Providence, RI (2013). <https://doi.org/10.1090/conm/588/11701>
- [95] Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983) <https://doi.org/10.1126/science.220.4598.671>
- [96] Wilcoxon, F.: Individual comparisons by ranking methods. *Biometrics Bulletin* **1**(6), 80–83 (1945) <https://doi.org/10.2307/3001968>
- [97] Holm, S.: A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* **6**(2), 65–70 (1979)
- [98] Hodges, J.L., Lehmann, E.L.: Estimates of location based on rank tests. *The Annals of Mathematical Statistics* **34**(2), 598–611 (1963) <https://doi.org/10.1214/aoms/1177704172>
- [99] Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. *Mathematical Programming* **91**(2), 201–213 (2002) <https://doi.org/10.1007/s101070100263>